CCA 3 - Practical Coding Assignment

Image Restoration, Segmentation, Feature Extraction, Object Recognition and Motion Estimation

Technical report - Computer Vision, Units 3, 4 and 5

Python | OpenCV | NumPy | Matplotlib | scikit-image | Ultralytics YOLO

Student name: Kritika Kumrawat

PRN / Roll number: 1272251096

Programme / Division: MCA E

Course instructor: Rahul Inchal

Submission date: 3 November 2026

Results generated by the project code; report built on 01 November 2026.

Yellow highlights mark details that the student must fill in or experiments that are still pending.

Contents

Contents 2
1. Assignment objective 4
1.1 Status of the experiments 4
2. Software and hardware environment 4
3. Input data 5
3.1 Original photograph 5
3.2 Second image for Unit 4 5
3.3 Original video (Unit 5) 6
3.4 Front-camera video (Unit 5A) 7
3.5 Supplementary data (clearly not the student's own) 7
4. Unit 3 - Image restoration and segmentation 7
4.1 Image restoration 7
4.2 Morphological processing and segmentation 12
5. Unit 4 - Feature extraction and image matching 16
5.1 Histogram of Oriented Gradients (HOG) 16
5.2 SIFT keypoints and descriptors 19
5.3 Feature matching 19
5.4 Matching after rotation, resizing and lighting changes 21
6. Unit 5 - Object recognition (Viola-Jones and YOLO) 24
6.1 Method 24
6.2 Results on the portrait 24
6.3 Missed and incorrect detections (observed) 25
6.4 Face-specific cascade versus general object detector 28
7. Unit 5 - Video motion estimation 28
7.1 Method 28
7.2 Results 29
8. Execution screenshots 31
8.1 Input check 31
8.2 Unit 3 - restoration and segmentation 32
8.3 Unit 4 - HOG and SIFT 39
8.4 Unit 5 - Haar cascade, YOLO and motion 43
8.5 Automated tests 49
9. Limitations and pending work 49
10. Conclusion 49
11. Git repository and explanation video 50
Appendix A - Source code 50
Appendix B - How to reproduce 50

1. Assignment objective

The objective is to implement important techniques from image restoration, image segmentation, feature extraction, object recognition and motion estimation on the student's own photograph and self-recorded video, and to compare the techniques using measured results. The work is organised as four tasks:

- **Task 1 (Unit 3):** image restoration (noise, Gaussian and median filtering, PSNR/SSIM) and morphological processing and segmentation (thresholding, erosion, dilation, opening, closing, Canny, watershed).
- **Task 2 (Unit 4):** HOG features, SIFT keypoints and descriptors, SIFT matching between two photographs, and matching after rotation and resizing.
- **Task 3 (Unit 5):** object recognition with a Viola-Jones Haar cascade and a pretrained YOLO model.
- **Task 4 (Unit 5):** video motion estimation with frame differencing, GMM background subtraction and Lucas-Kanade optical flow.

1.1 Status of the experiments

Table 1. *Status of each task in the latest run (from outputs/run_summary.json).*

Task	Name	Status	Pending items
3A	Image restoration	COMPLETED	-
3B	Morphology and segmentation	COMPLETED	-
4	HOG and SIFT feature matching	COMPLETED	-
5A	Haar cascade and YOLO detection	COMPLETED	-
5B	Video motion estimation	COMPLETED	-

2. Software and hardware environment

Table 2. *Environment used to produce the results.*

Item	Value
Operating system	Windows 11 (CoreSingleLanguage), build 26300
Processor	AMD Ryzen 5 5625U with Radeon Graphics
Memory (RAM visible to Windows)	7.3 GB
GPU	No CUDA GPU - all inference on the CPU
Python	3.11.18 (conda-forge build), in the project virtual environment .venv

Table 3. *Python libraries, installed versions and their role in the project.*

Package	Version	Role
opencv-python	4.14.0.94	Image I/O, colour conversion, filtering, thresholding, morphology, Canny, cv2.watershed, SIFT, BF/FLANN matching, RANSAC, Haar cascades, MOG2, Lucas-Kanade, video read/write
numpy	2.4.6	Image arrays, synthetic noise with a seeded random generator, statistics
scikit-image	0.26.0	PSNR/SSIM metrics, HOG features and visualisation, peak detection and watershed, sample data
scipy	1.17.1	Hole filling and connected-component labelling (scipy.ndimage)
matplotlib	3.11.2	All comparison figures and charts
ultralytics	8.4.191	Pretrained YOLO11n object detector (COCO, 80 classes)
torch	2.14.1	Deep-learning runtime used by Ultralytics (CPU build)
pillow	13.0.0	Writing the animated GIF preview of the motion results
python-docx	1.2.0	Generating this report
pytest	9.1.1	Automated tests

The project runs from a single entry point, main.py, which runs each unit and writes all images, figures, CSV tables and JSON result files to the outputs folder. All synthetic noise and random operations use seed 42, so repeated runs give the same numbers (timings vary slightly).

3. Input data

3.1 Original photograph

File inputs/original_photo.jpg: 1233 x 1275 pixels, 3 colour channels, JPEG, 63 KB. The original file is kept unchanged; every result is written to the outputs folder.

Content: a head-and-shoulders portrait of the student in a dark navy blazer and a light striped shirt, photographed against a plain light grey-blue backdrop. It contains one person and one frontal face, large smooth regions (backdrop, blazer) and fine detail (hair strands, eyes, teeth, shirt stripes). This mix makes it suitable for noise and restoration tests, thresholding, edge detection, SIFT and face detection. It is not suitable for splitting touching objects, so a supplementary image is used for that watershed experiment (Section 4.2).

The copy used here has no camera metadata (EXIF), so capture device and date cannot be read from the file.



Figure 1. *Original photograph used as the clean reference image (unmodified input).* *[inputs/original_photo.jpg]*

3.2 Second image for Unit 4

File inputs/second_photo.png: 463 x 609 pixels. A second, separately taken photograph of the student (close-up, indoor shopping street with hanging lanterns), supplied by the student for Unit 4. The file received is a small PNG without camera metadata (a screenshot or exported copy), not the phone's original image.



Figure 2. *Second photograph of the student (Unit 4).* *[inputs/second_photo.png]*

File inputs/second_photo_b.png: 277 x 399 pixels. A wider photograph taken in the same place as the second photograph, used as an extra same-scene matching pair.



Figure 3. *Photograph B, same scene as the second photograph (wider view).* *[inputs/second_photo_b.png]*

3.3 Original video (Unit 5)

Table 4. *Properties of the self-recorded video (read by OpenCV).*

Property	Value
File	inputs/original_video.mp4
Resolution	832 x 464
Frame rate	59.92 fps
Frames	1479
Duration	24.7 s
File size	5.0 MB
Processed at	640x357

Video description: Recorded by the student with a phone held still in one position. The scene is a room with a white wall, an orange wall with a window, a purple curtain, a bed and a blue box on the floor. The student walks into the frame, crosses the room towards and away from the camera, and leaves the frame several times; for some seconds nobody is visible. The file was shared through WhatsApp, which re-encodes videos, so it is not the camera's original file.

3.4 Front-camera video (Unit 5A)

File inputs/front_camera_video.mp4: 576 x 1024 pixels, 30 fps, 5.9 minutes, recorded by the student with the phone's front camera (head and shoulders, talking to the camera). Only frames sampled from it are used, for Haar and YOLO face detection (Section 6.3); it is not used for motion estimation. It was also shared through WhatsApp.

3.5 Supplementary data (clearly not the student's own)

- **Watershed sample:** scikit-image's bundled 'coins' photograph (Greek coins from Pompeii, Brooklyn Museum collection, no known copyright restrictions) - used for watershed on multiple objects, because the portrait has none.
- **Synthetic teaching image:** a computer-generated image of three overlapping discs, used to show how watershed splits touching objects.
- **Preliminary matching view:** a tilted, re-lit and slightly blurred copy of the original photograph, generated by the code, used only until the second photograph is supplied.
- **Tests:** scikit-image's public astronaut image and synthetic images/clips are used only inside the automated tests.

4. Unit 3 - Image restoration and segmentation

4.1 Image restoration

Method

- **Gaussian noise:** zero-mean Gaussian noise with standard deviation 25 grey levels added to every pixel and channel, then clipped to 0-255.
- **Salt-and-pepper noise:** 5% of pixel positions set to pure white or pure black (half each).
- **Gaussian filter:** cv2.GaussianBlur - a weighted average of the neighbourhood (sigma derived from the kernel size).
- **Median filter:** cv2.medianBlur - each pixel replaced by the median of its neighbourhood.
- **Kernels:** kernel sizes 3, 5, 7, 9, 11; figures show 5x5.
- **Metrics:** PSNR = 10 log10(255^2 / MSE) in dB, and SSIM (mean structural similarity over the three colour channels), both from scikit-image, computed against the clean original at the same size and in RGB order. Higher values mean the image is closer to the original.

Results



Figure 4. *Gaussian noise (sigma 25) and the two 5x5 restorations, with PSNR/SSIM.*
[outputs/unit3/restoration/fig_gaussian_noise_comparison.png]



Figure 5. *Salt-and-pepper noise (5%) and the two 5x5 restorations.* *[outputs/unit3/restoration/fig_salt_pepper_comparison.png]*

Table 5. *Gaussian noise: PSNR and SSIM of the noisy and restored images against the original (restoration_metrics.csv).*

Method	Kernel	PSNR (dB)	SSIM	Time (ms)
Noisy (no filter)	-	20.44	0.131	0.0
Gaussian filter	3x3	28.87	0.497	3.6
Gaussian filter	5x5	31.40	0.661	3.1
Gaussian filter	7x7	33.44	0.795	1.4
Gaussian filter	9x9	34.21	0.852	1.5
Gaussian filter	11x11	34.52	0.888	1.7
Median filter	3x3	27.89	0.446	3.4
Median filter	5x5	31.75	0.701	5.8
Median filter	7x7	33.65	0.827	140.1
Median filter	9x9	34.37	0.882	130.6
Median filter	11x11	34.42	0.907	132.4

Table 6. *Salt-and-pepper noise: PSNR and SSIM of the noisy and restored images against the original (restoration_metrics.csv).*

Method	Kernel	PSNR (dB)	SSIM	Time (ms)
Noisy (no filter)	-	17.98	0.223	0.0
Gaussian filter	3x3	26.18	0.451	1.5
Gaussian filter	5x5	28.55	0.583	2.2
Gaussian filter	7x7	30.47	0.719	1.3
Gaussian filter	9x9	31.27	0.789	1.4

Method	Kernel	PSNR (dB)	SSIM	Time (ms)
Gaussian filter	11x11	31.73	0.839	1.7
Median filter	3x3	47.43	0.995	1.4
Median filter	5x5	44.03	0.987	5.8
Median filter	7x7	41.43	0.978	92.7
Median filter	9x9	39.72	0.970	104.4
Median filter	11x11	38.41	0.964	102.4

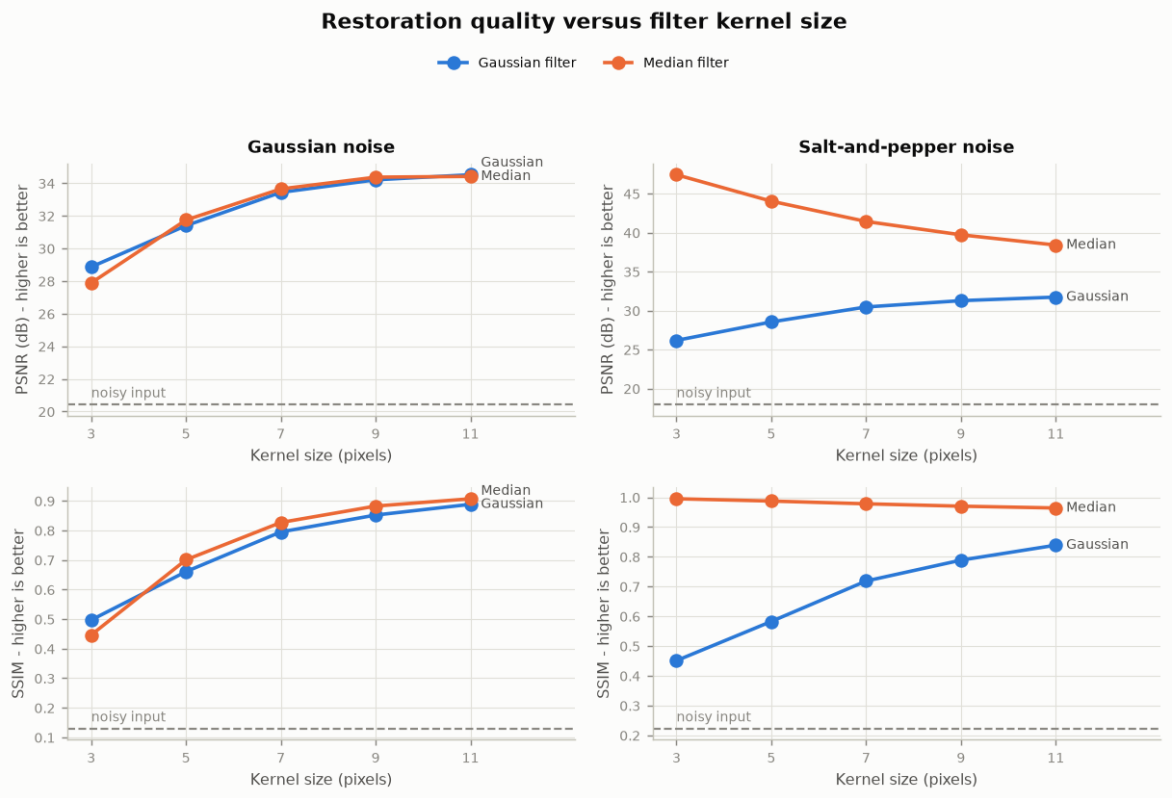


Figure 6. *PSNR and SSIM against kernel size; the dashed line is the noisy input.* *[outputs/unit3/restoration/fig_psnr_ssim_vs_kernel.png]*

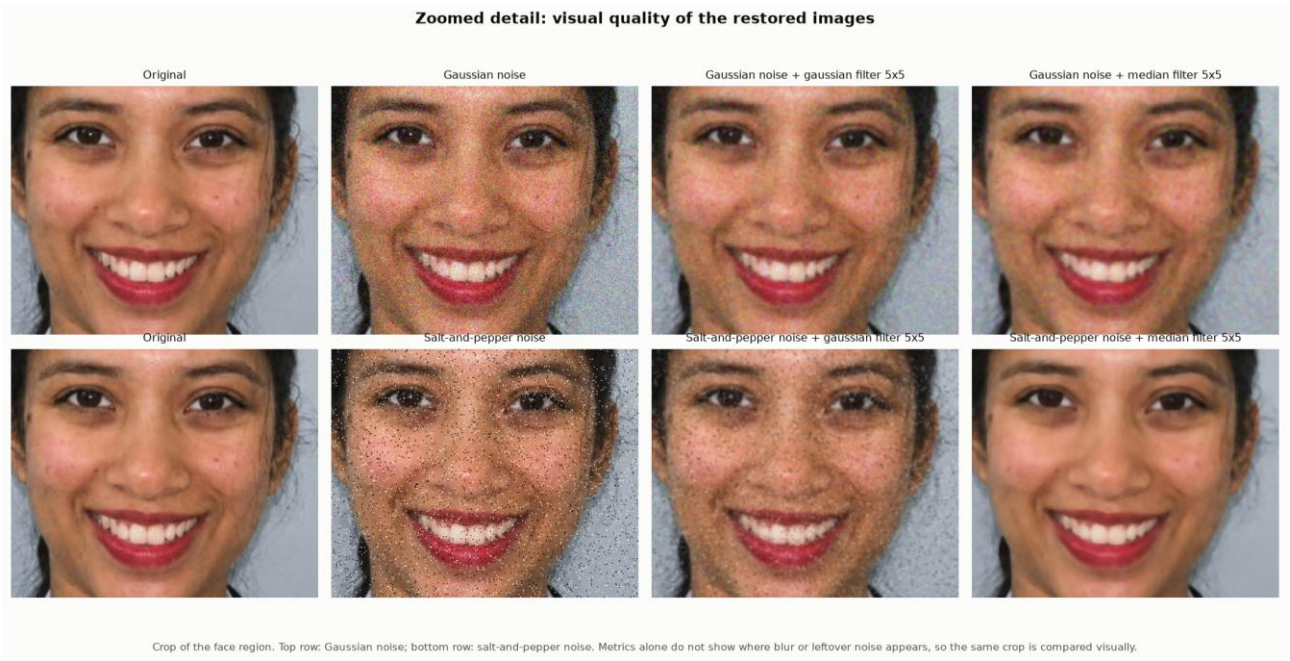


Figure 7. *Zoomed face region for visual comparison (5x5 kernels).* *[outputs/unit3/restoration/fig_zoomed_face_comparison.png]*



Figure 8. *Kernel-size trade-off: small, medium and large kernels on the same crop.* *[outputs/unit3/restoration/fig_kernel_size_tradeoff.png]*

Comparison and observations

Table 7. *Gaussian versus median filtering - summary of the measured results.*

Noise type	Noisy input	Best Gaussian filter	Best median filter	Better technique
Gaussian noise	20.44 dB / 0.131	11x11: 34.52 dB / 0.888	11x11: 34.42 dB / 0.907	Similar: best PSNR gaussian filter, best SSIM median filter
Salt-and-pepper noise	17.98 dB / 0.223	11x11: 31.73 dB / 0.839	3x3: 47.43 dB / 0.995	Median filter (large margin)

- **Impulse noise:** Salt-and-pepper noise lowered the image to 17.98 dB / SSIM 0.223. The 3x3 median filter restored it to 47.43 dB / SSIM 0.995, while the best Gaussian filter reached only 31.73 dB. A median picks one of the real neighbouring values, so isolated black and white pixels are simply discarded; a Gaussian filter averages them into their neighbours, leaving grey specks that are visible in the zoomed figure.
- **Kernel size:** For the median filter, larger kernels lowered PSNR and SSIM on salt-and-pepper noise because the noise is already removed at 3x3 and bigger windows only remove real detail.
- **Gaussian noise:** Gaussian noise lowered the image to 20.44 dB / SSIM 0.131. Both filters improved it by similar amounts: at 5x5 the Gaussian filter gave 31.40 dB and the median 31.75 dB. The best PSNR was gaussian filter 11x11 (34.52 dB) and the best SSIM median filter 11x11 (0.907).
- **Metrics versus visual quality:** The metrics kept rising up to the largest kernel because most of this portrait is smooth (backdrop, blazer, skin), where heavy smoothing is harmless. The kernel-size figure shows the cost: large kernels also soften eyelashes, hair strands and skin texture. PSNR and SSIM summarise the whole image, so a visual check of detailed regions is still needed when choosing a kernel.
- **Speed:** the median filter took 1-6 ms for 3x3 and 5x5 kernels but 93-140 ms for larger kernels on this image, because OpenCV uses a fast sorting network only for kernels up to 5x5 and a histogram-based method above that. The Gaussian filter stayed fast at every size.

4.2 Morphological processing and segmentation

Thresholding

The portrait was converted to grayscale and smoothed with a 5x5 Gaussian blur. Otsu's method chose a global threshold of 102; the mask is inverted so that dark regions (hair, eyebrows, eyes, the blazer) become white foreground. Adaptive thresholding (block 31, C = 5) compares each pixel with its own neighbourhood and keeps local texture and edges instead of large regions.

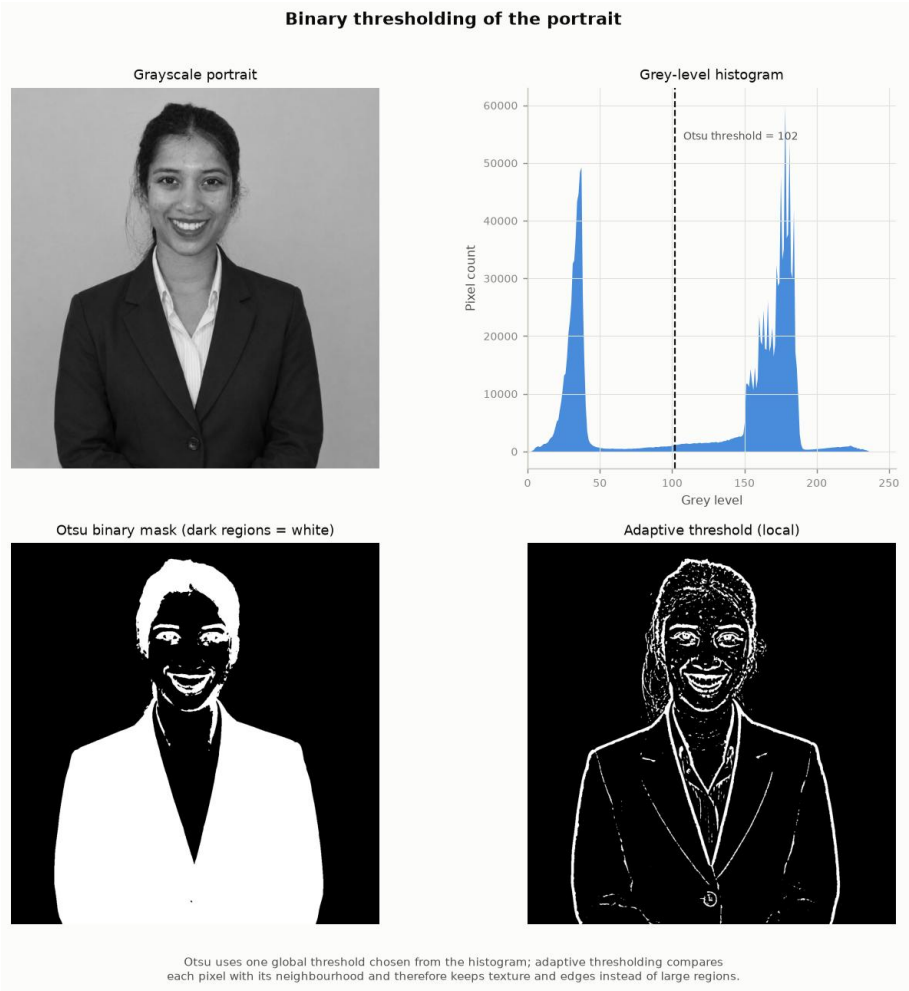


Figure 9. *Grayscale portrait, histogram with the Otsu threshold, and the two binary masks.*
[outputs/unit3/segmentation/fig_thresholding.png]

Erosion, dilation, opening and closing

Table 8. *Effect of each operation on the Otsu mask (9x9 ellipse structuring element).*

Operation	Foreground (%)	White blobs	Blobs < 50 px	Enclosed holes
Binary mask (Otsu)	35.83	37	26	30
Erosion	33.75	22	10	4
Dilation	38.08	14	0	12
Opening	35.52	9	0	8
Closing	36.02	25	16	4

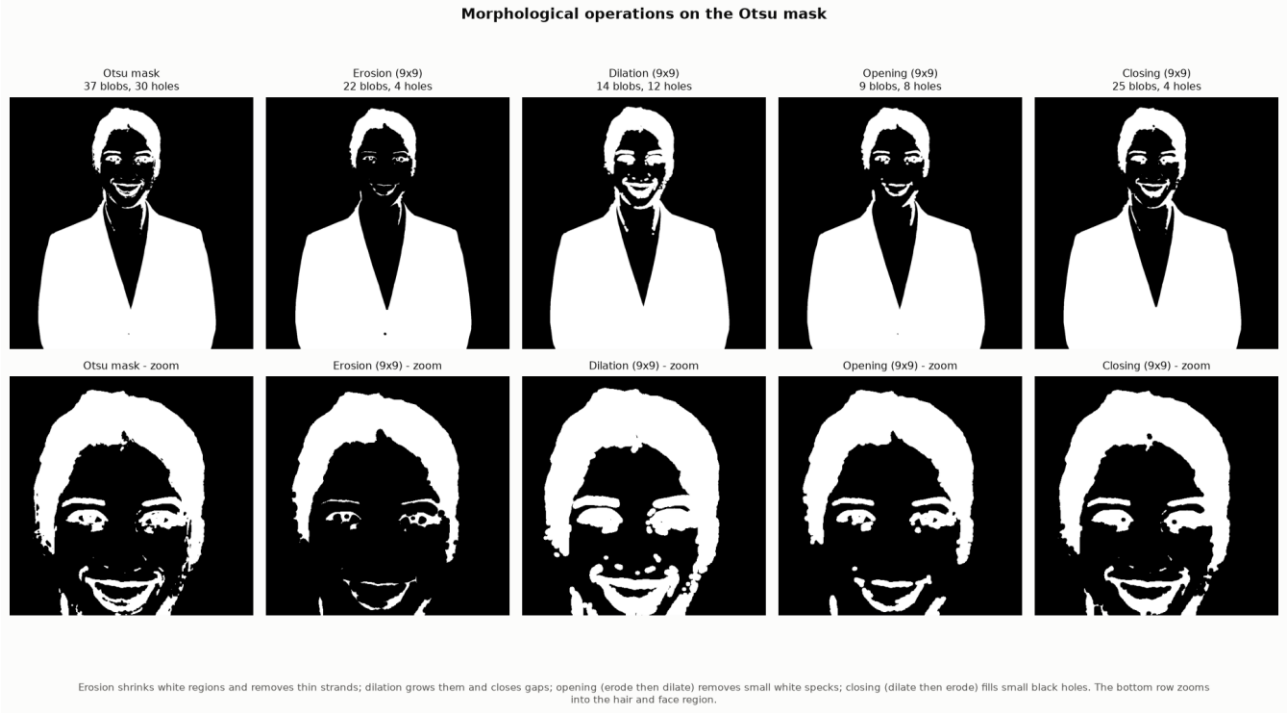


Figure 10. *Morphological operations; bottom row zooms into the hair and face.* *[outputs/unit3/segmentation/fig_morphology.png]*

- **Measured:** erosion shrank the foreground from 35.83% to 33.75% and removed thin hair strands; dilation grew it to 38.08% and merged nearby features such as the eyes and eyebrows.
- **Measured:** opening reduced the number of white blobs from 37 to 9 (small specks removed) while keeping the foreground almost unchanged (35.52%); closing reduced enclosed holes from 30 to 4.
- **Best use:** opening is the better choice for removing small noise blobs, closing for filling small gaps inside regions; plain erosion and dilation change object size and are mainly useful as building blocks.

Canny edge detection

Table 9. *Canny edge statistics (canny_statistics.csv).*

Input	Low / high threshold	Edge pixels (%)
Clean grayscale	50 / 150	0.56
Clean grayscale (automatic thresholds)	107 / 214	0.37
Gaussian-noisy grayscale	50 / 150	0.75
Gaussian-noisy after 5x5 median filter	50 / 150	0.44

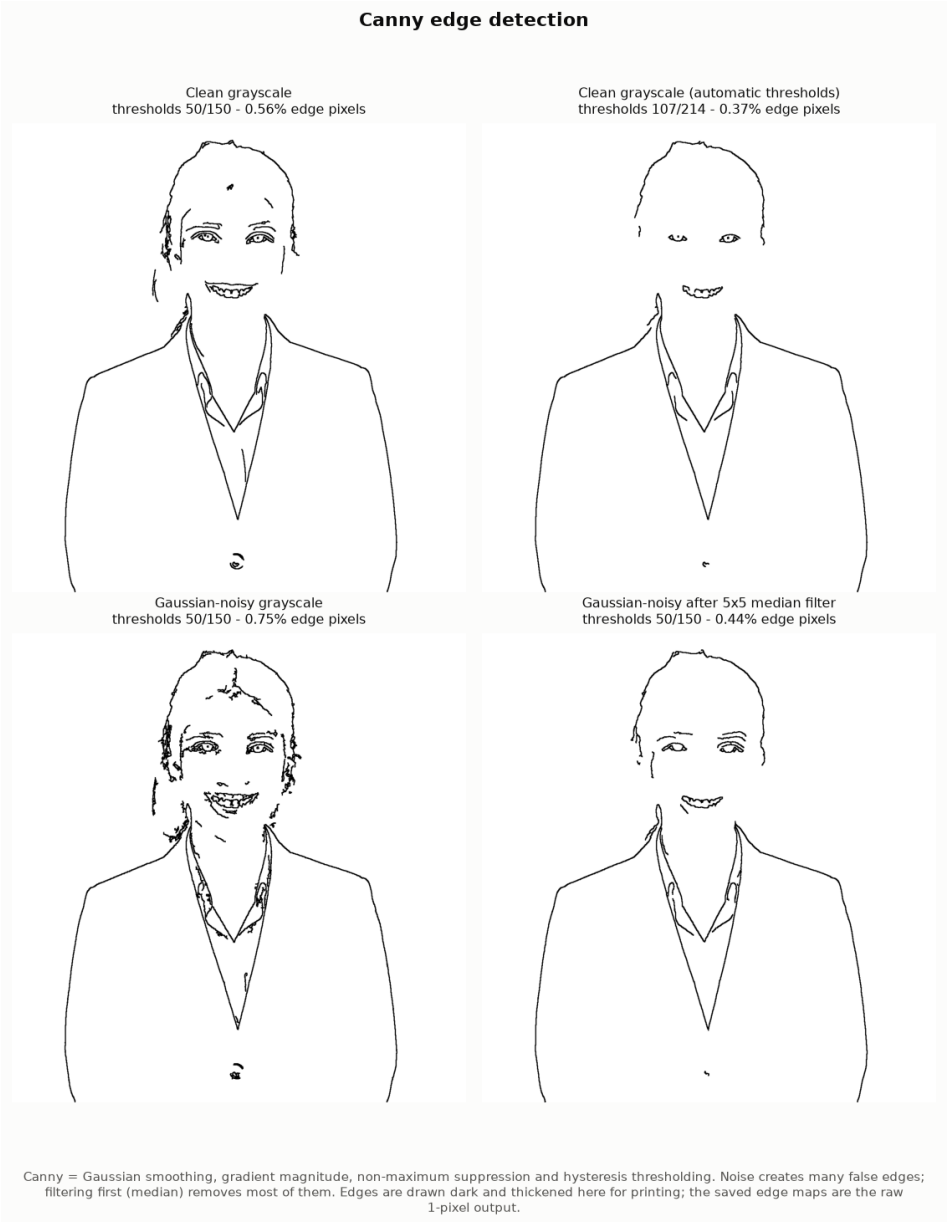


Figure 11. *Canny edges on the clean, noisy and median-restored portrait.* *[outputs/unit3/segmentation/fig_canny.png]*

On the clean portrait 0.56% of pixels are edges (hairline, eyes, mouth, collar and lapels). Gaussian noise raised this to 0.75% because noise creates false gradients; a 5x5 median filter before Canny brought it back to 0.44%. Restoration therefore matters for later segmentation steps. The automatic thresholds (median-based) gave 0.37% edge pixels.

Watershed segmentation

Watershed treats the image like a landscape and floods it from markers; where floods from two markers meet, a boundary is drawn. For touching objects the markers are the peaks of the distance transform (the centre of each object), so two touching round objects get two markers and are separated.

The portrait shows one person and no touching objects, so the touching-objects experiment uses a clearly labelled supplementary image (scikit-image 'coins', 24 coins). Uneven lighting in that photo was first removed with a morphological top-hat, then the mask was thresholded with Otsu, cleaned with opening and closing, and split with distance-transform watershed.

Table 10. *Watershed results.*

Image	Connected blobs after thresholding	Markers	Watershed regions
scikit-image sample 'coins'	22	24	24

Image	Connected blobs after thresholding	Markers	Watershed regions
Synthetic overlapping discs	1	3	3

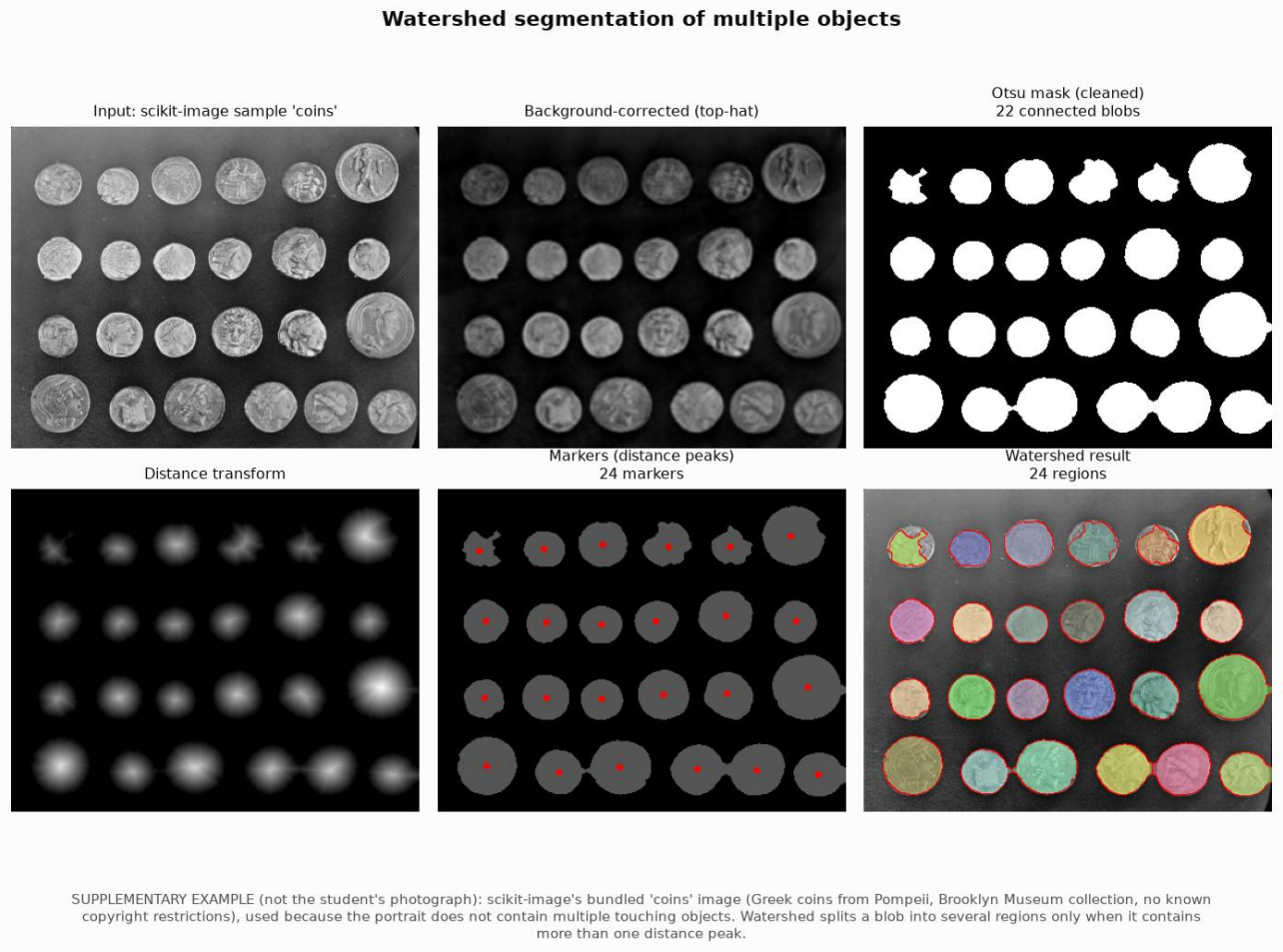


Figure 12. *Distance-transform watershed on the supplementary coins image (not the student's photograph).*
[outputs/unit3/segmentation/fig_watershed_objects.png]

Result: the cleaned mask had 22 blobs because closing joined some neighbouring coins in the bottom row with thin bridges; the distance transform still had one peak per coin, so watershed produced 24 regions and split the bridged coins. The darkest coin (top-left) has low contrast, so its outline is irregular - a limitation of thresholding, not of watershed.

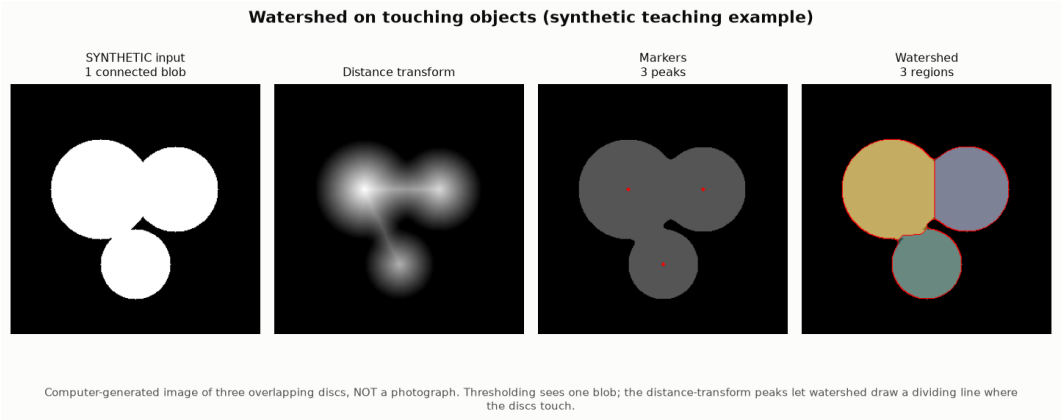


Figure 13. *Synthetic teaching example: one connected blob split into three regions.*
[outputs/unit3/segmentation/fig_watershed_touching_synthetic.png]

On the portrait itself, marker-based watershed (cv2.watershed) separated the person from the plain backdrop: markers came from the colour distance to the backdrop colour learned from the image border. The person mask covers 45.0% of the image. This is foreground/background segmentation, not separation of touching objects.

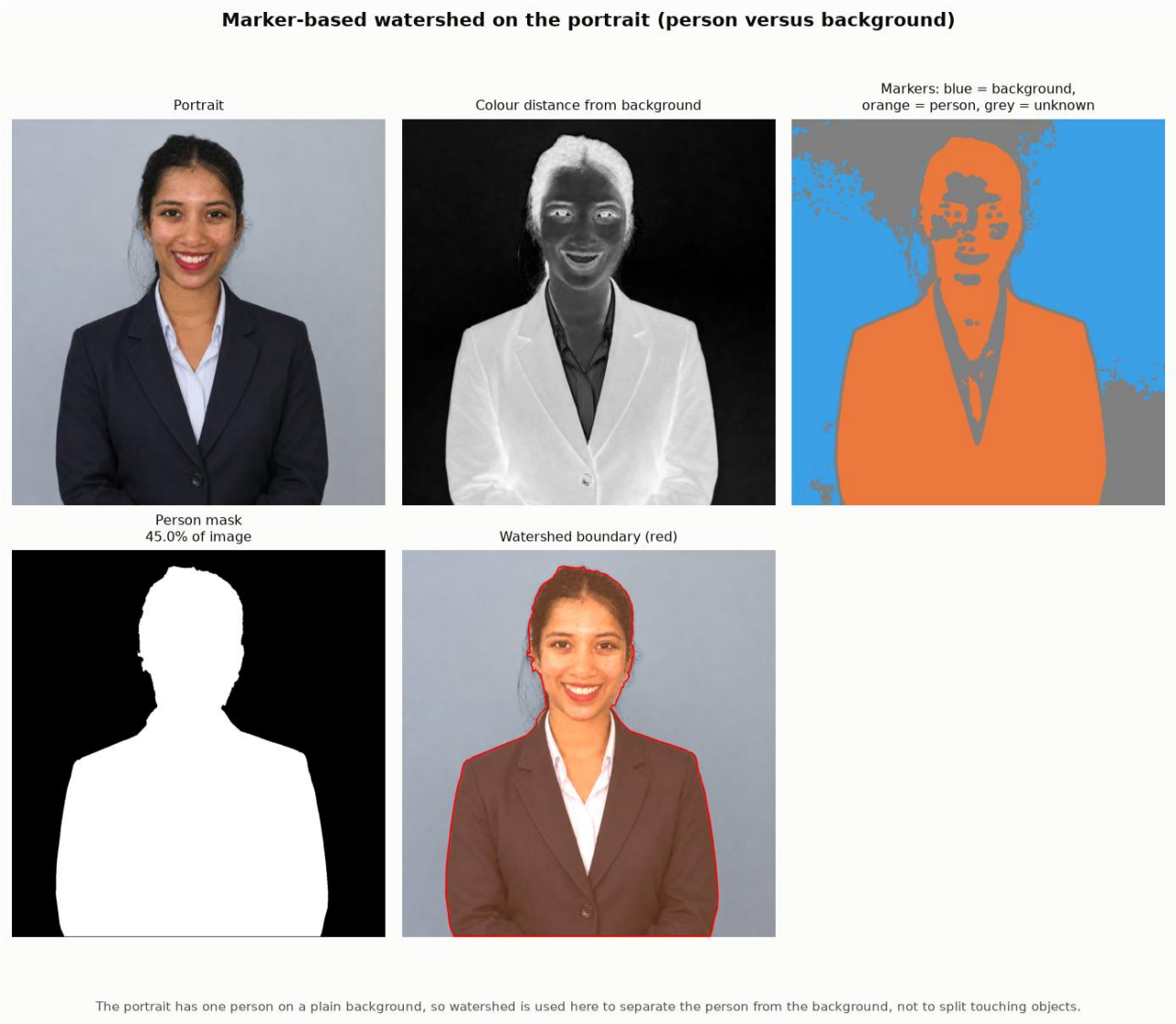


Figure 14. *Marker-based watershed on the portrait.* *[outputs/unit3/segmentation/fig_watershed_portrait.png]*

Table 11. *Morphological operations and segmentation methods - comparison.*

Technique	What it produced on the selected image	Works best for
Otsu threshold	Global split at 102: hair, eyes and blazer as foreground	Images with two clear brightness groups
Adaptive threshold	Local texture and outlines	Uneven lighting, documents, texture
Erosion / dilation	Foreground 33.75% / 38.08%	Shrinking / growing shapes
Opening / closing	Blobs 37 -> 9; holes 30 -> 4	Removing specks / filling holes
Canny	0.56% edge pixels (clean)	Thin, well-located boundaries
Watershed (distance)	24 regions on scikit-image sample 'coins'	Touching round objects
Watershed (markers)	Person mask 45.0% of the portrait	Object vs plain background

5. Unit 4 - Feature extraction and image matching

5.1 Histogram of Oriented Gradients (HOG)

HOG divides the image into small cells, computes the gradient direction and strength of each pixel, and builds a 9-bin histogram of directions per cell. Groups of 2x2 cells (blocks) are normalised (L2-Hys) so that the descriptor is robust to lighting. The portrait was resized to 512 px on its longer side so the cells are visible.

Table 12. *HOG feature length for different cell sizes (hog_statistics.csv).*

Image	HOG input	Cell size	Cells	Feature length	Expected length	Time (ms)
Original photo	495x512	8 px	61x64	136,080	136,080	394
Original photo	495x512	16 px	30x32	32,364	32,364	102

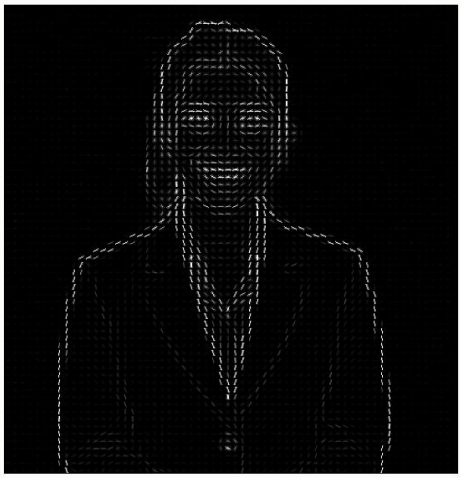
Image	HOG input	Cell size	Cells	Feature length	Expected length	Time (ms)
Original photo	495x512	32 px	15x16	7,560	7,560	43
Second photo	389x512	8 px	48x64	106,596	106,596	252
Second photo	389x512	16 px	24x32	25,668	25,668	79
Second photo	389x512	32 px	12x16	5,940	5,940	34

Histogram of Oriented Gradients (HOG)

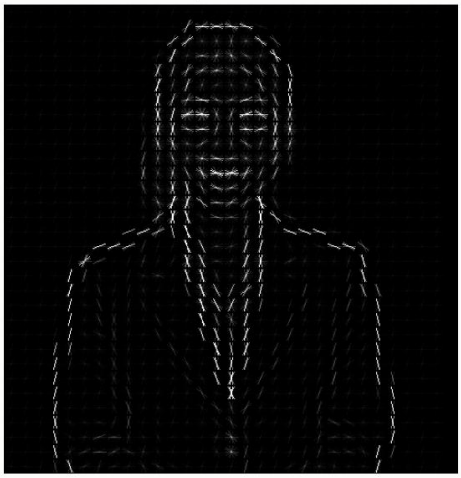
Grayscale input (495x512)



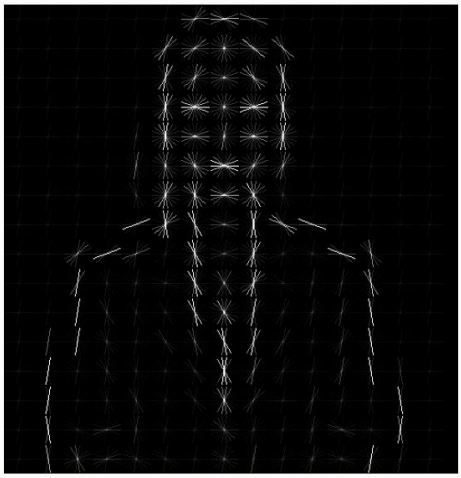
HOG, 8x8-pixel cells
136,080 features



HOG, 16x16-pixel cells
32,364 features



HOG, 32x32-pixel cells
7,560 features



Second photo (grayscale)



Second photo HOG, 16x16 cells

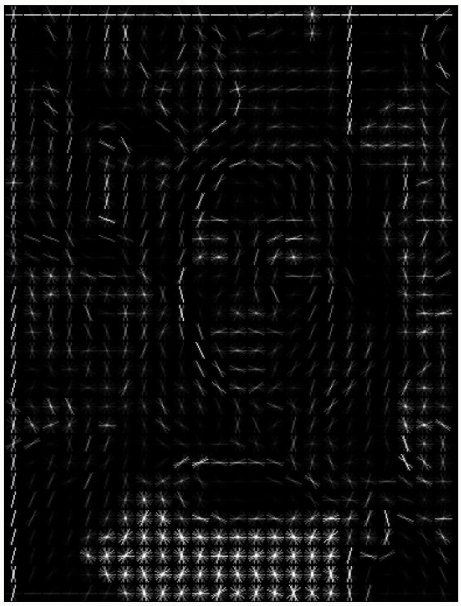


Figure 15. HOG visualisation with 8, 16 and 32-pixel cells. [outputs/unit4/fig_hog.png]

With 16-pixel cells the HOG picture clearly outlines the head, hairline, shoulders and lapels; 8-pixel cells also show the eyes and mouth but produce a vector 4.2 times longer; 32-pixel cells keep only the coarse silhouette.

Every measured length equals blocks across x blocks down x cells per block x orientations, which confirms the configuration.

5.2 SIFT keypoints and descriptors

Table 13. SIFT keypoints (sift_keypoints.csv).

Image	Size	Keypoints	Descriptor matrix	Time (ms)
Original photo	1233x1275	545	545x128	185
Second photo	463x609	1677	1677x128	46

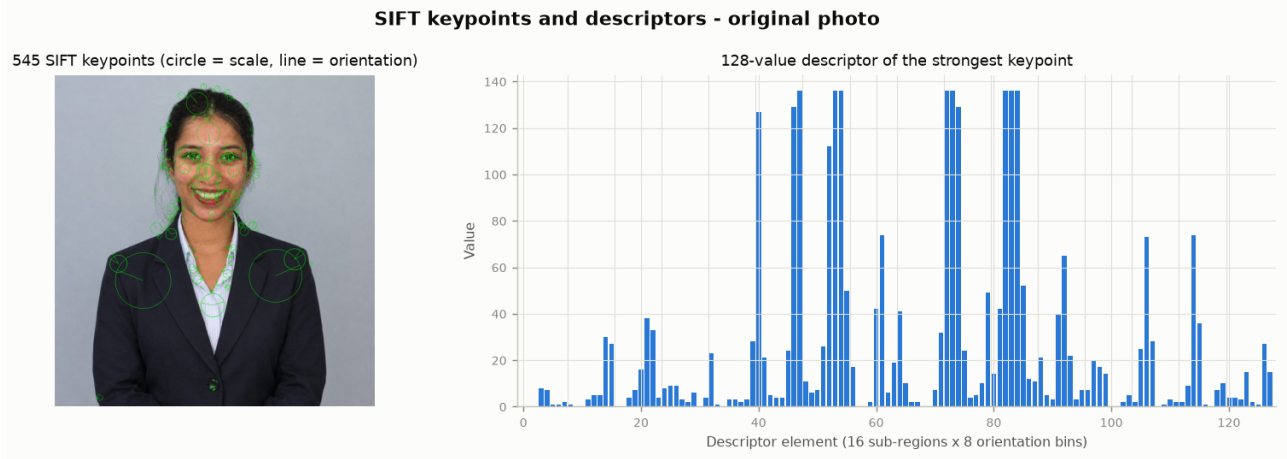


Figure 16. SIFT keypoints (circle size = scale, line = orientation) and the 128-value descriptor of the strongest keypoint. [outputs/unit4/fig_sift_keypoints.png]

SIFT found 545 keypoints on the portrait. They concentrate on textured, high-contrast areas (hair, eyes, mouth, collar and shirt stripes); the smooth backdrop and the plain blazer give very few. Each keypoint has a 128-value descriptor: a 4x4 grid of sub-regions around the point, each with an 8-bin histogram of gradient directions, measured relative to the keypoint's own orientation and scale.

5.3 Feature matching

Descriptors were matched with a brute-force matcher (L2 distance, two nearest neighbours) and Lowe's ratio test (keep a match only if its distance is below 0.75 x the second-best distance). RANSAC then counted the matches that agree with one geometric model (homography for transformed copies of one image, fundamental matrix for two real photographs). For transformed copies the true transform is known, so a match is counted as correct when it lands within 3 px of the true position.

Table 14. Matching between the two images (pair_matching.csv).

Pair	Keypoints 1 / 2	Ratio-test matches	Match ratio	RANSAC inliers	Correct
Original vs second photo	545 / 1677	19	3.5%	9 (47.4%)	n/a (unknown)
Second photo vs second photo B (same scene)	1677 / 985	14	1.4%	8 (57.1%)	n/a (unknown)

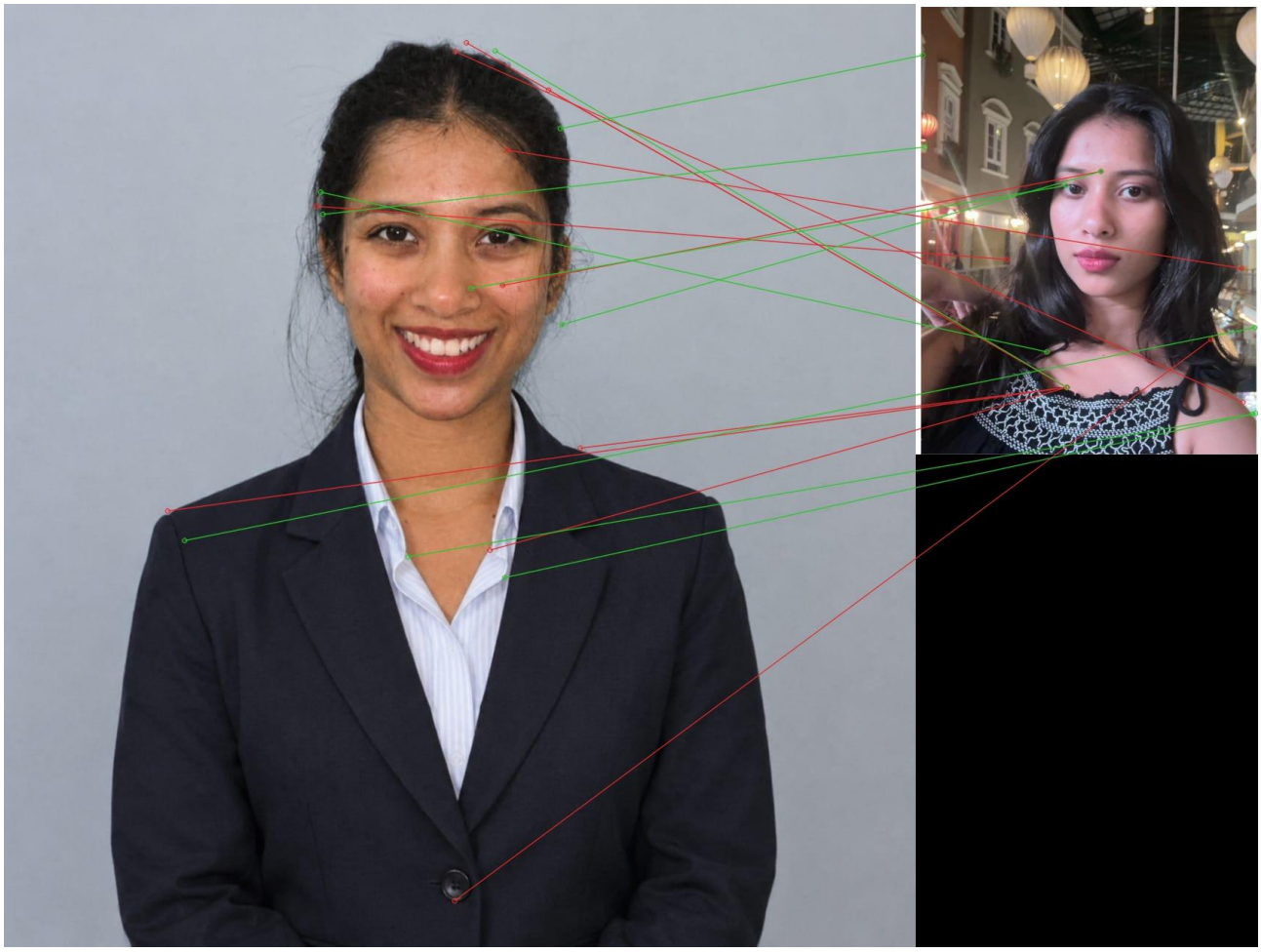


Figure 17. *Matches between the original and the second photograph.* *[outputs/unit4/04_matches_original_vs_second.jpg]*

Only 19 matches passed the ratio test and 9 agreed with the RANSAC model. With so few points a geometric model (a fundamental matrix needs at least 8) can be fitted to wrong matches by chance, so these inliers are not reliable evidence of correspondence.

Visual check of the match picture: none of the drawn lines, including the green RANSAC inliers, connects the same feature in both photographs (for example, eye to hair or blazer button to shoulder). SIFT therefore found no true correspondences between the portrait and the second photograph. This is expected: SIFT matches local texture, and the two photos differ in place, clothing, hairstyle, lighting and camera, so the only shared content is the face, seen at a different size, pose and expression.

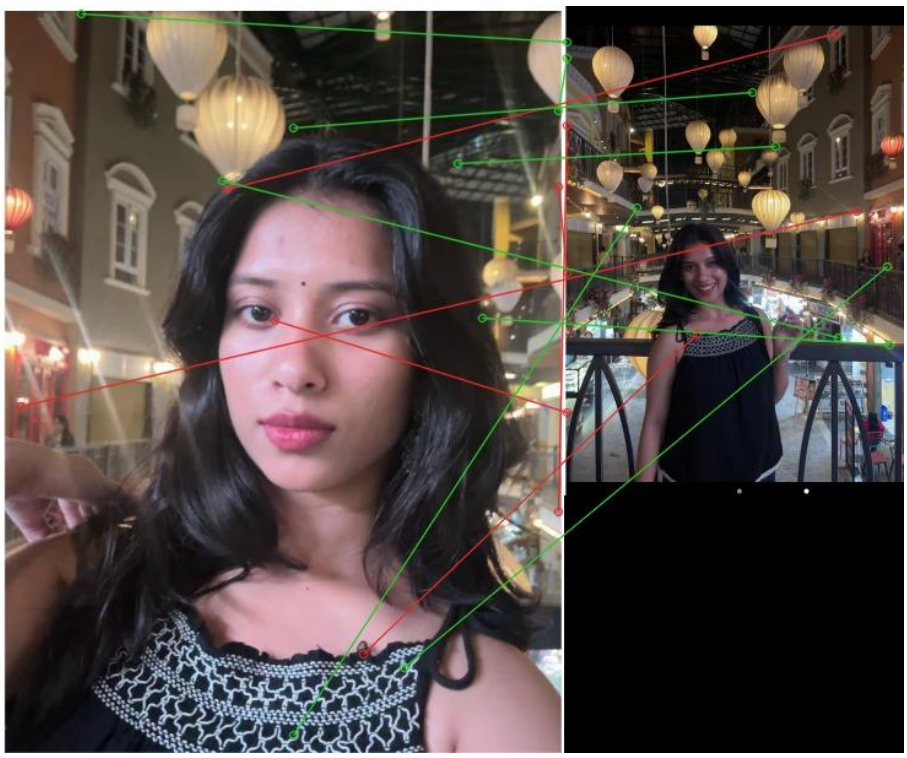


Figure 18. *Second photograph vs photograph B of the same scene: 14 matches, 8 RANSAC inliers.*
[outputs/unit4/04b_matches_second_vs_second_b_same_scene.jpg]

For the two photos taken in the same place, a few lines join one lantern to another, but the lanterns look alike, so it cannot be confirmed that they are the same lanterns; most other lines join unrelated features. The photos are small (277 x 399 and 463 x 609 pixels), one is a close-up and the other a wide view (a large scale change), and repeated objects such as the lanterns make the ratio test reject or confuse matches. Full-resolution originals from the phone, or two photos taken from closer positions, would give far more reliable matching.

Table 15. *Brute-force versus FLANN matcher on the same pair (matcher_comparison.csv).*

Matcher	Ratio-test matches	RANSAC inliers	Matching time (ms)
Brute force (BFMatcher, L2)	19	9	4.0
FLANN (KD-tree, 5 trees, 50 checks)	20	10	17.8

The two matchers kept 19 and 20 matches. Matching took 4.0 ms with brute force and 17.8 ms with FLANN, so brute force was faster here. FLANN is an approximate search (with an index-building cost) that pays off for large descriptor sets; with a few hundred keypoints the exact brute-force search is sufficient.

5.4 Matching after rotation, resizing and lighting changes

Table 16. *Original photograph versus transformed copies of itself (transform_matching_original.csv).*

Transform	Image 2 size	Keypoints 2	Ratio-test matches	RANSAC inliers	Correct (%)	Median error (px)
Rotated 15 deg	1521x1551	490	373	364	97.6	0.14
Rotated 30 deg	1705x1721	501	387	371	95.1	0.22
Rotated 45 deg	1773x1773	498	386	373	96.4	0.30
Rotated 90 deg	1275x1233	565	512	512	99.4	0.50
Rotated 180 deg	1233x1275	554	510	502	97.8	0.71
Resized x0.25	308x319	128	120	102	84.2	0.29
Resized x0.5	616x638	227	200	183	91.0	0.20
Resized x0.75	925x956	397	314	307	96.5	0.14
Resized x1.5	1850x1912	554	426	415	96.2	0.20
Resized x2	2466x2550	551	459	456	98.9	0.36
Gamma 0.5 (brighter)	1233x1275	373	261	244	92.3	0.15
Gamma 2 (darker)	1233x1275	537	296	281	91.9	0.15

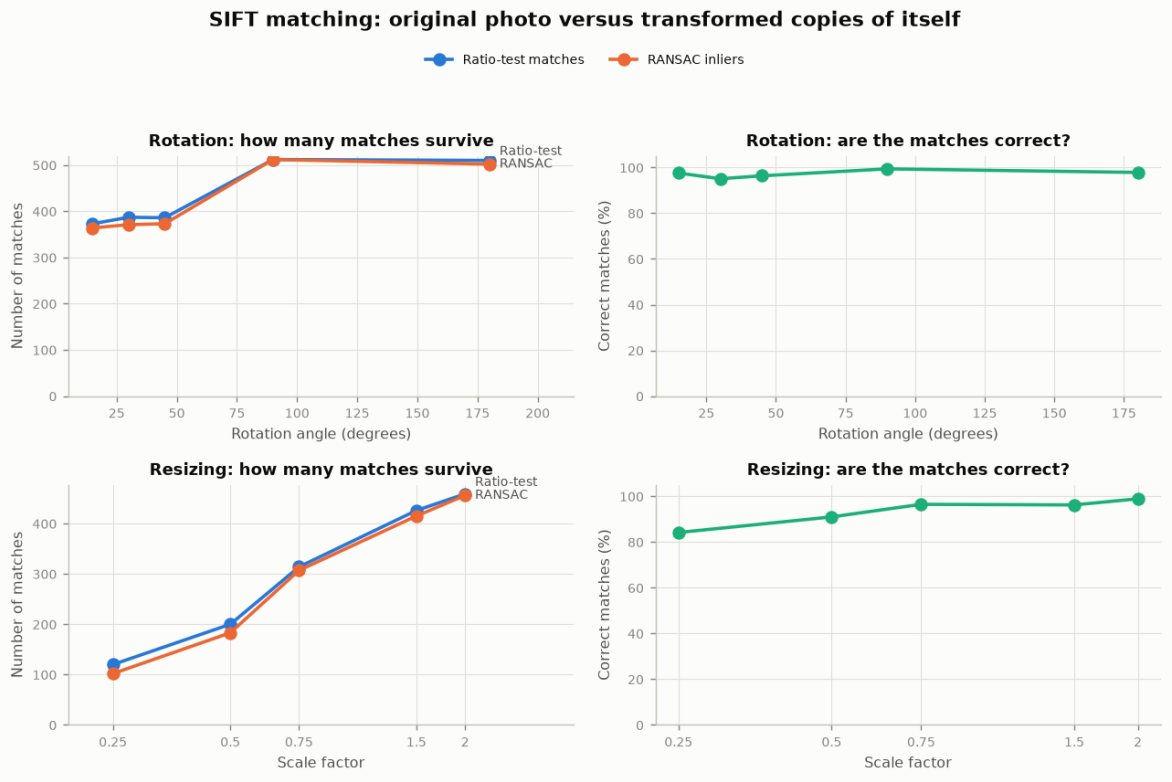


Figure 19. *Matches and correct-match rate against rotation angle and scale.* *[outputs/unit4/fig_rotation_scale_sweep.png]*



Figure 20. *Original versus a copy rotated by 45 degrees.* *[outputs/unit4/06_matches_original_vs_rotation.jpg]*

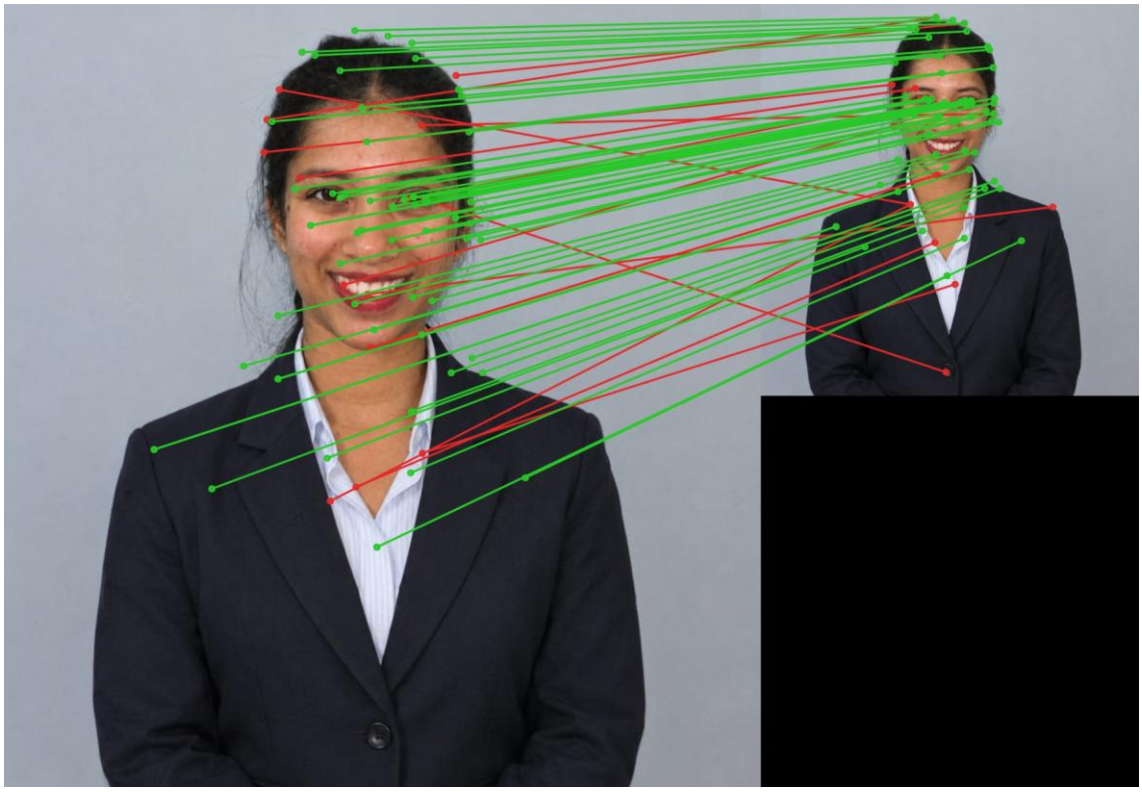


Figure 21. *Original versus a copy resized to 0.5x.* *[outputs/unit4/06_matches_original_vs_resize.jpg]*



Figure 22. *Original versus a copy darkened (gamma 2).* *[outputs/unit4/06_matches_original_vs_illumination.jpg]*

Observations

- **Rotation:** every rotation kept at least 95% correct matches, because SIFT measures each descriptor relative to the keypoint's dominant orientation. Rotations by 90 and 180 degrees kept 510-512 matches, but 15-45 degrees kept only 373-387: right-angle rotations move pixels exactly, while other angles need interpolation, which slightly changes local gradients.
- **Scale:** matches fell from 459 to 120 at 0.25x, mainly because the small image has only 128 keypoints (fine detail disappears), but 84.2% of the remaining matches were still correct thanks to SIFT's scale-space search.
- **Illumination:** gamma 0.5 (brighter): 261 matches, 92.3% correct; gamma 2 (darker): 296 matches, 91.9% correct. The geometry is unchanged, so the matches that survive are almost all correct, but fewer survive: descriptors are normalised gradient histograms and tolerate uniform brightness scaling, whereas a non-linear gamma change compresses contrast in the bright (gamma < 1) or dark (gamma > 1) range, so weak gradients there fall below SIFT's contrast threshold or change shape.

- **Viewpoint and scene change:** between the two real photographs SIFT kept only 19 ratio-test matches (9 RANSAC inliers, which the visual check shows are not true correspondences), compared with hundreds for rotated or resized copies of one image. The two photos differ in scene, clothing, hairstyle, lighting, camera and resolution, so almost no local patch is shared. SIFT is invariant to rotation and scale, not to a change of scene.

Table 17. *Original versus the second image before and after rotating/resizing the second image.*

Experiment	Ratio-test matches	Match ratio	RANSAC inliers
Original vs second photo	19	3.5%	9
Original vs second photo rotation 45	10	1.8%	7
Original vs second photo rotation 90	20	3.7%	9
Original vs second photo resize 0.5	13	2.4%	8
Original vs second photo resize 1.5	21	3.9%	10

6. Unit 5 - Object recognition (Viola-Jones and YOLO)

6.1 Method

- **Haar cascade:** OpenCV's pretrained haarcascade_frontalface_default.xml (and haarcascade_eye.xml inside each face). The Viola-Jones detector slides windows over an image pyramid and evaluates simple light/dark rectangle (Haar-like) features with an integral image; boosted stages reject non-faces early. Parameters: scaleFactor 1.1, minNeighbors 5, minimum face size 8% of the short image side, after histogram equalisation.
- **YOLO:** Ultralytics yolo11n.pt pretrained on COCO (80 classes), run on the CPU with confidence threshold 0.25. A convolutional network predicts boxes, class labels and confidence scores for the whole image in one forward pass.

6.2 Results on the portrait

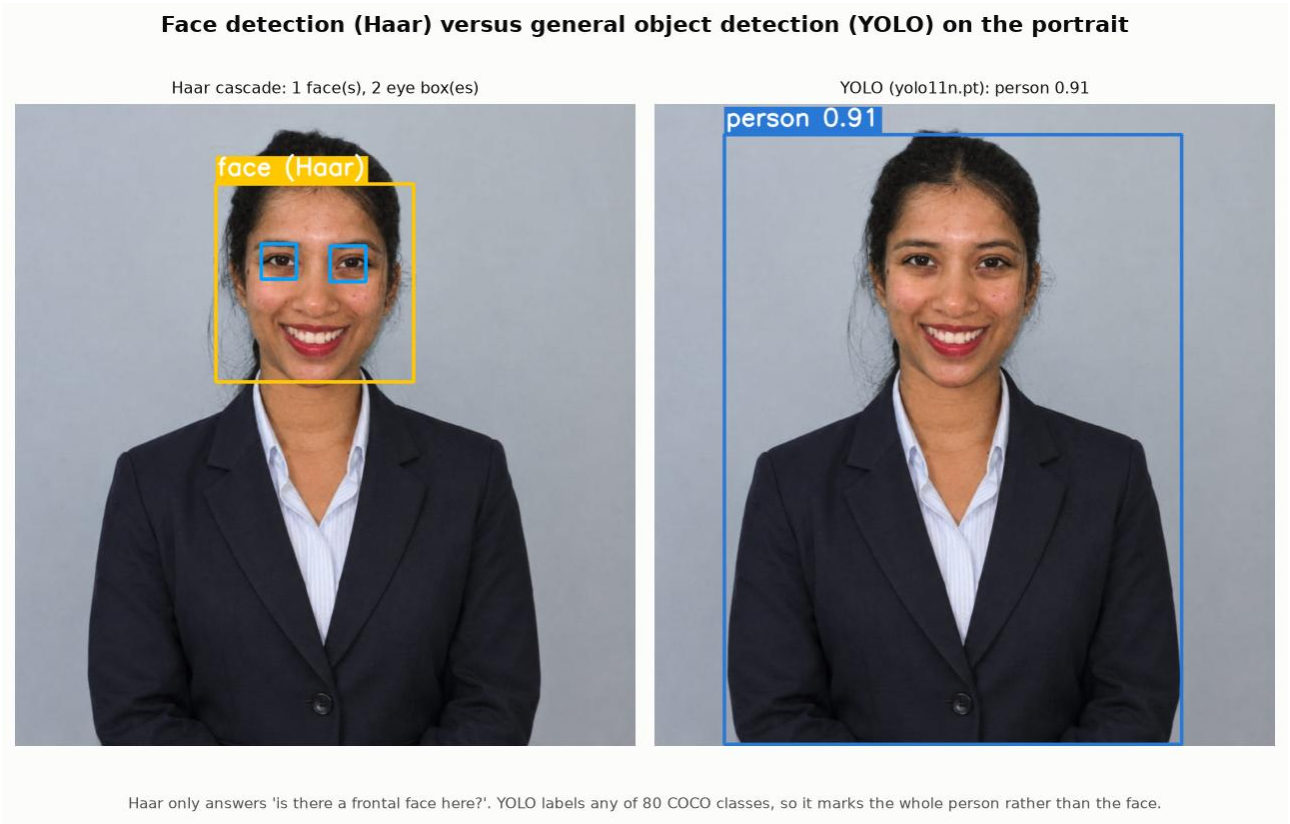


Figure 23. *Haar cascade (amber, eyes in light blue) versus YOLO (blue) on the portrait.*
[outputs/unit5/detection/fig_haar_vs_yolo_portrait.png]

Table 18. Haar cascade versus YOLO (haar_vs_yolo_comparison.csv).

Aspect	Haar cascade	YOLO
What it detects	One object type per cascade file (here: frontal faces, eyes)	80 COCO classes (person, chair, phone, ...) in one pass
Output	Boxes only (no class score)	Boxes + class label + confidence score
Portrait result	1 face box(es), 2 eye box(es)	person 0.91
Time on this CPU (portrait)	20 ms (face cascade)	60 ms (after warm-up)
Model size / training	Small XML file shipped with OpenCV; boosted rectangle features	Neural network weights downloaded once; trained on COCO

On the upright portrait the Haar cascade returned 1 face box(es) and 2 eye box(es) in 20 ms; YOLO returned: person (0.91) in 60 ms (both timed after one warm-up run).

6.3 Missed and incorrect detections (observed)

Table 19. Haar cascade - effect of minNeighbors on the portrait (one face present).

minNeighbors	Boxes returned	Extra (false) boxes
1	5	4
2	2	1
3	1	0
5	1	0
8	1	0

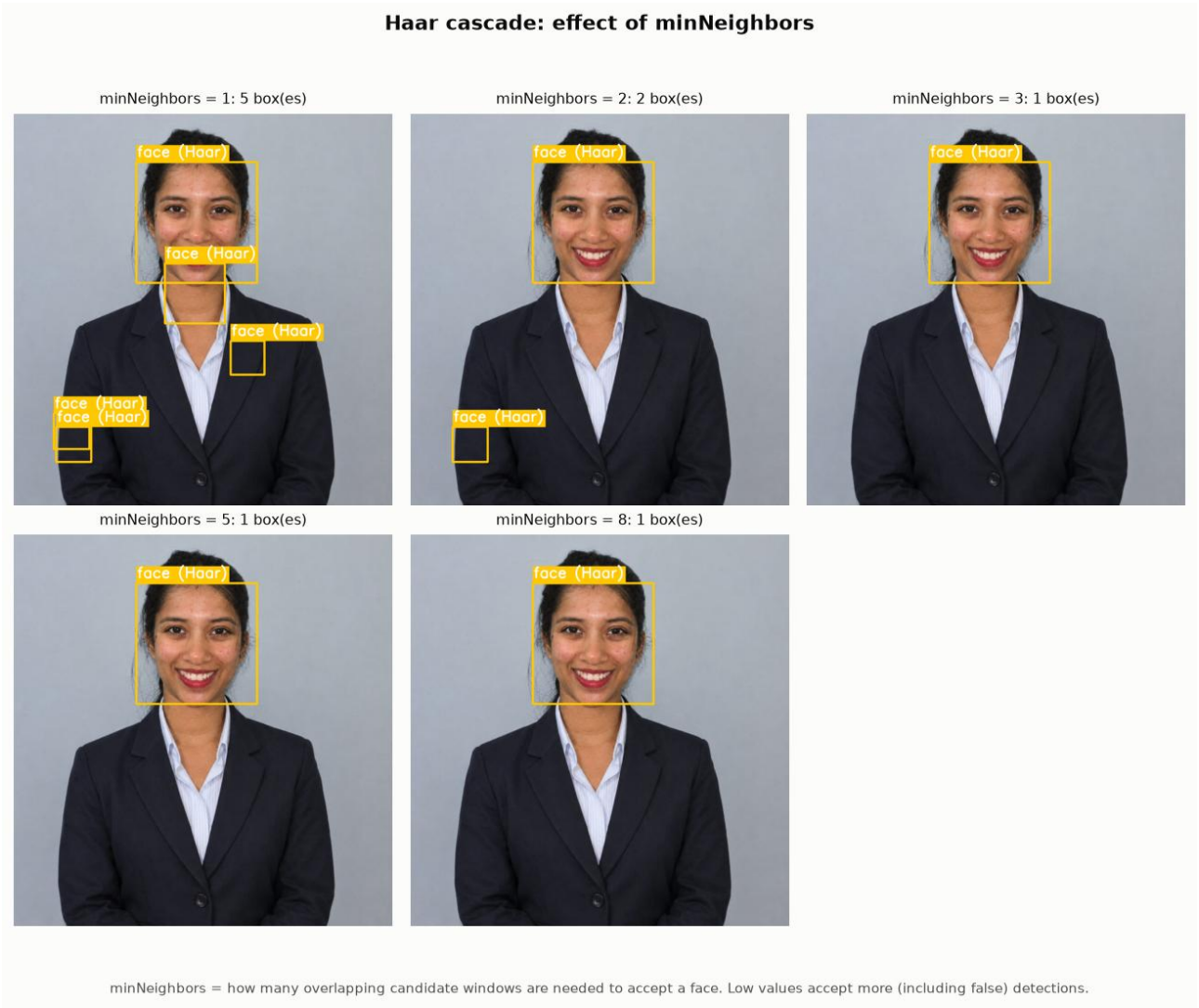


Figure 24. Haar detections for different minNeighbors values. [outputs/unit5/detection/fig_haar_min_neighbors.png]

Table 20. Detections on rotated copies of the portrait (one face and one person always present).

Rotation	Haar faces	YOLO persons	Best person confidence	Other YOLO labels
0 deg	1	1	0.91	-
15 deg	1	1	0.91	-
30 deg	0	1	0.90	-
45 deg	0	1	0.89	-
90 deg	0	2	0.50	-

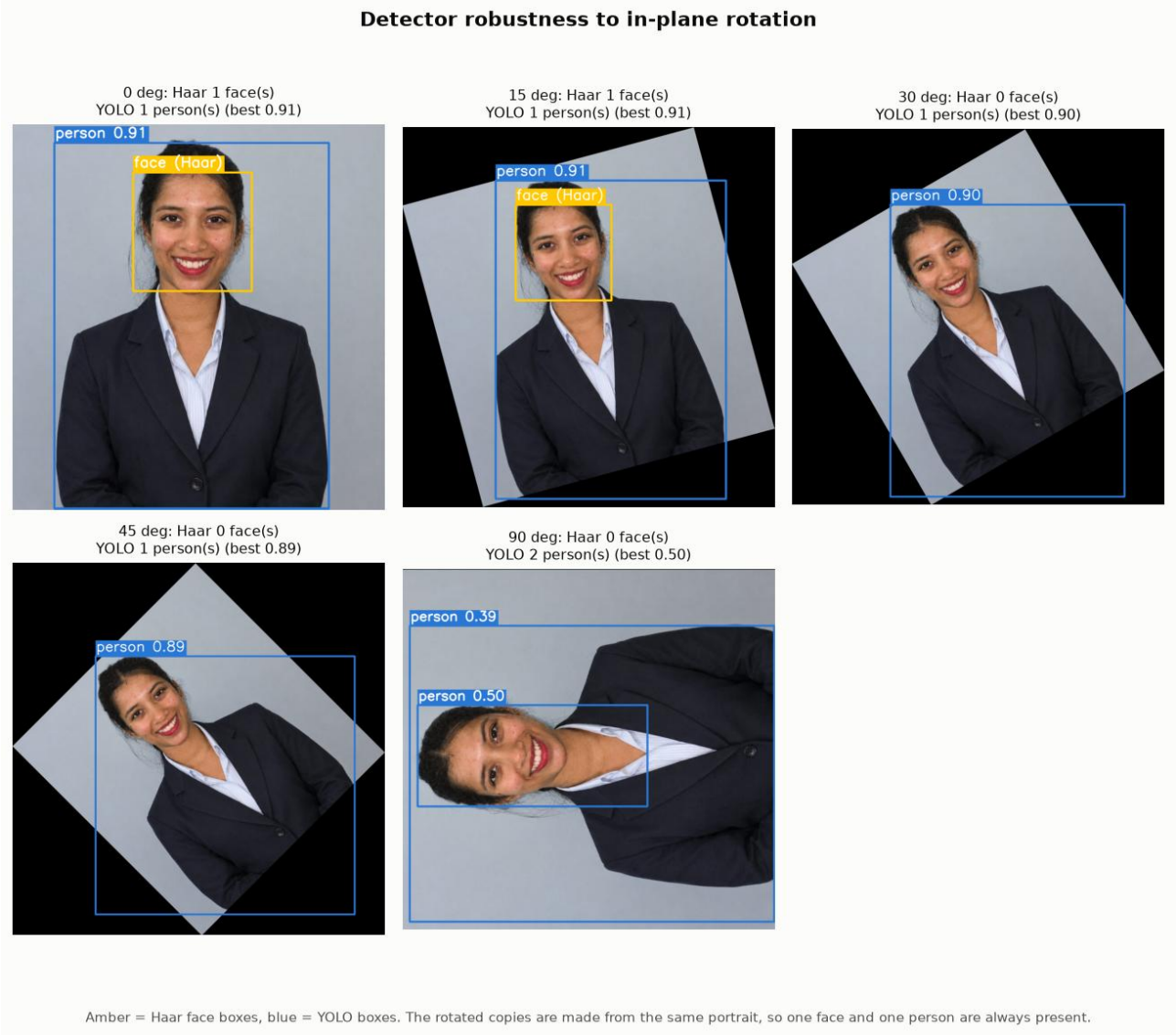


***Figure 25.** Haar (amber) and YOLO (blue) on rotated copies. [outputs/unit5/detection/fig_rotation_robustness.png]*

Observations recorded automatically from the measurements:

- Haar cascade with minNeighbors=1 returned 5 boxes for a portrait that contains 1 face: at least 4 false detection(s).
- Haar cascade with minNeighbors=2 returned 2 boxes for a portrait that contains 1 face: at least 1 false detection(s).
- Missed detection: the Haar cascade found 0 face(s) in the portrait rotated by 30 degrees, although the face is fully visible.
- Missed detection: the Haar cascade found 0 face(s) in the portrait rotated by 45 degrees, although the face is fully visible.
- Missed detection: the Haar cascade found 0 face(s) in the portrait rotated by 90 degrees, although the face is fully visible.

- Duplicate/false detection: YOLO returned 2 person boxes in the portrait rotated by 90 degrees (one person present).
- Extra detection: the Haar cascade returned 2 face boxes in front-camera frame 5272 (175.7 s), which shows one face.
- Extra detection: the Haar cascade returned 2 face boxes in front-camera frame 7380 (246.0 s), which shows one face.
- Incorrect eye detections: in 3 of 5 front-camera frames the eye cascade returned more than two eye boxes per face (up to 5 in one frame); the extra boxes fall on other dark facial features such as the nostrils or mouth.

Why the Haar cascade fails from 30 degrees: the frontal-face cascade was trained on upright faces, so its fixed rectangle features no longer line up with the eyes, nose and mouth once the face is tilted. With a low minNeighbors value it also accepts isolated windows on textured areas.

YOLO learned from varied photographs and found the person at every tested angle, but at 90 degrees its confidence dropped to 0.50 and it returned an extra, overlapping person box (a duplicate detection).



Figure 26. *Haar and YOLO on frames of the original video.* *[outputs/unit5/detection/fig_video_frame_detections.png]*

Table 21. *Detections on video frames.*

Frame	Haar faces	YOLO detections
295	0	person 0.91, bed 0.86
739	0	bed 0.74
1183	0	person 0.86, bed 0.36

In the sampled frames the student is seen from behind or in side profile, or is out of view, so the frontal-face Haar cascade found no face; in the profile frame the face is visible, which counts as a missed detection. YOLO detected the person whenever she was in view and also labelled the bed.

Frames of the front-camera video

The student also recorded a front-camera (selfie) clip. Five frames sampled evenly through it were used for the same two detectors; this is the situation the frontal-face cascade is designed for.

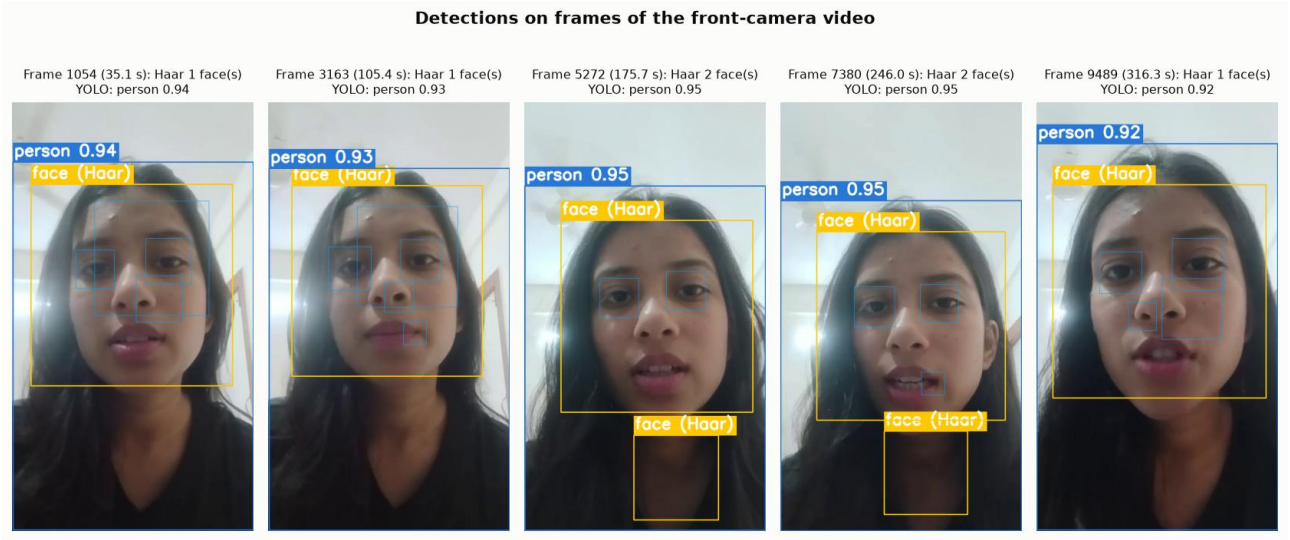


Figure 27. *Haar faces (amber), eyes (light blue) and YOLO (blue) on frames of the front-camera video.*
[outputs/unit5/detection/fig_front_camera_detections.png]

Table 22. *Detections on front-camera frames (front_camera_frame_detections.csv).*

Frame	Time (s)	Haar faces	Haar eye boxes	YOLO detections
1054	35.1	1	4	person 0.94
3163	105.4	1	5	person 0.93
5272	175.7	2	2	person 0.95
7380	246.0	2	3	person 0.95
9489	316.3	1	4	person 0.92

The Haar cascade found the face in 5 of 5 frames, but in 2 of them it also returned a second, false face box lower in the image (on the neck and T-shirt). The eye cascade often returned more than two boxes per face, marking other dark features such as the nostrils or mouth as eyes. YOLO labelled the person in every frame. Compared with the walking video, where the face was seen from the side or behind, this shows how much the Haar cascade depends on a frontal, upright face.

6.4 Face-specific cascade versus general object detector

A Haar cascade answers a single yes/no question - 'is this window a frontal face?' - so it needs one cascade file per object type, gives no confidence score and is sensitive to rotation, pose and lighting, but it is tiny and fast on a CPU. YOLO is a general-purpose detector: one network localises and classifies 80 object classes at once with a confidence score, and is far more robust to pose and rotation, at the cost of a larger model, a deep-learning runtime and (on some machines) more computation. YOLO's COCO classes include 'person' but not 'face', so for face-level tasks a face-specific model is still needed.

7. Unit 5 - Video motion estimation

7.1 Method

- **Frame extraction:** frames are read with cv2.VideoCapture, resized so the longer side is at most 640 px, and processed in order; each pair of consecutive frames is compared.
- **Frame differencing:** absolute difference of consecutive blurred grayscale frames, threshold 25, small opening and dilation; contours give moving regions.
- **GMM background subtraction:** cv2.createBackgroundSubtractorMOG2 (history 300, varThreshold 16, shadow detection on; shadow pixels are removed from the mask). Each pixel is modelled as a mixture of Gaussians; pixels that do not fit the background components are foreground.
- **Lucas-Kanade optical flow:** up to 200 Shi-Tomasi corners tracked with cv2.calcOpticalFlowPyrLK (window 21x21, 3 pyramid levels), re-detected every 10 frames; points moving more than 1.0 px/frame are drawn as arrows.

7.2 Results



Figure 28. *Consecutive frames and their differences.* *[outputs/unit5/motion/fig_consecutive_frames.png]*

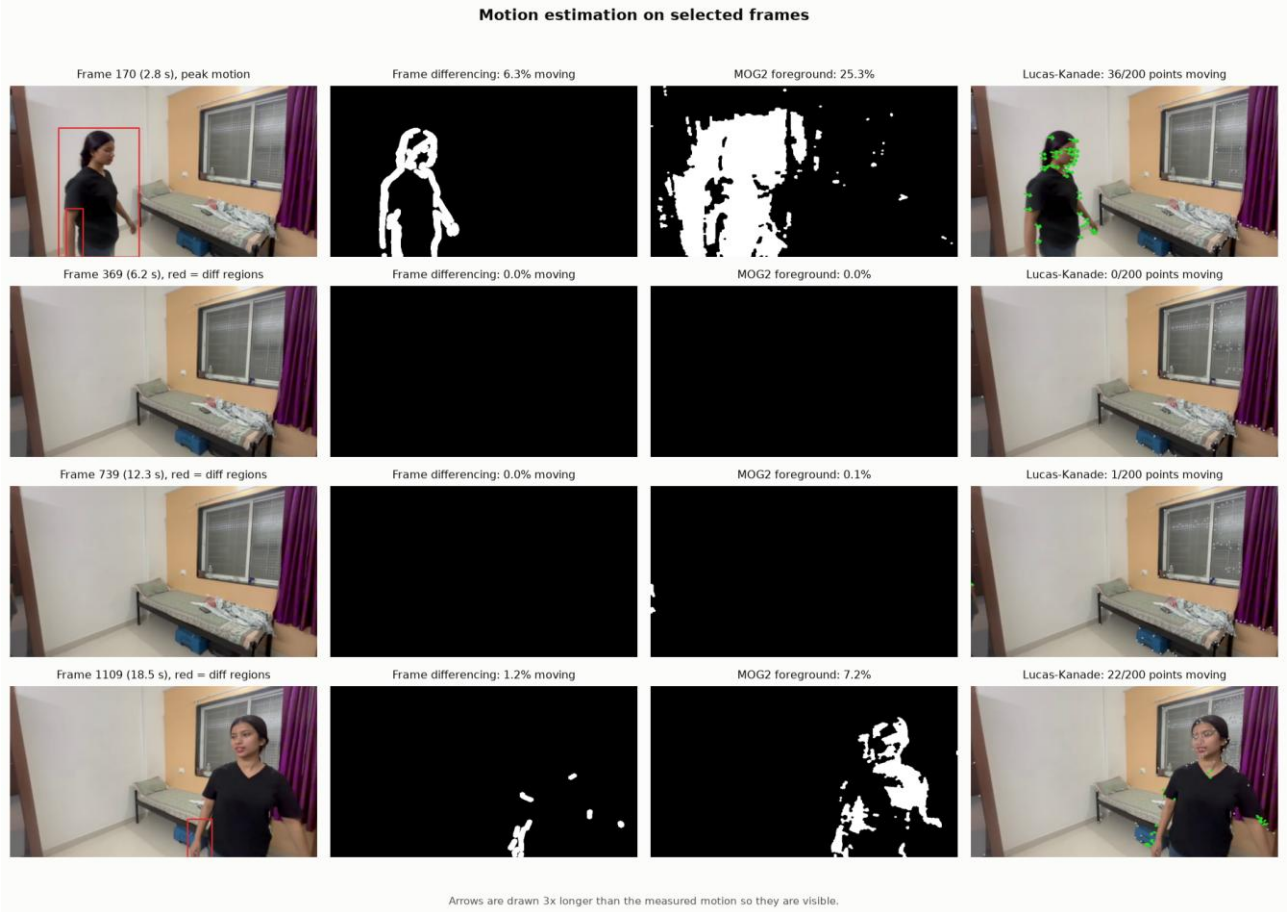


Figure 29. *Foreground masks and motion vectors on selected frames.* *[outputs/unit5/motion/fig_motion_snapshots.png]*

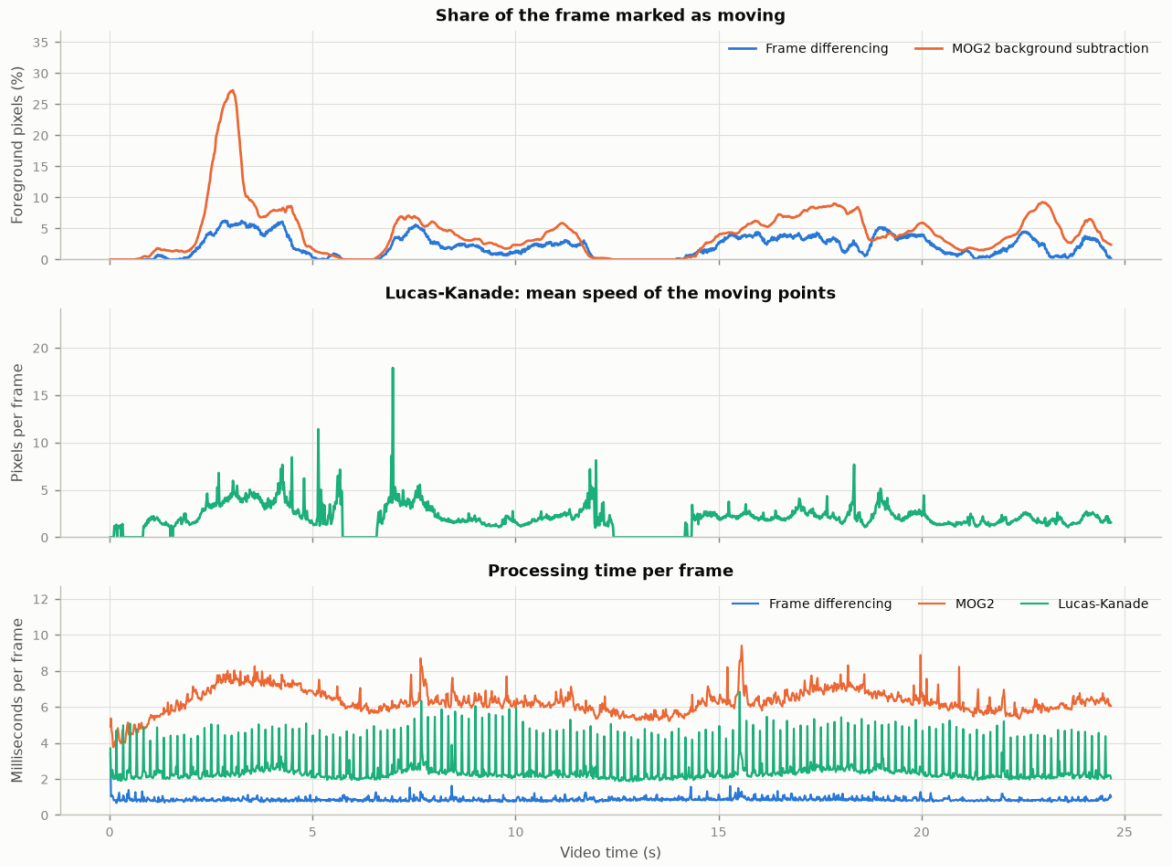


Figure 30. *Per-frame foreground share, Lucas-Kanade speed and processing time.* *[outputs/unit5/motion/fig_motion_timeseries.png]*

Table 23. *Measured comparison over the whole video (motion_comparison.csv).*

Technique	Mean foreground (%)	Mean regions / frame	Mean ms / frame
Frame differencing	2.07	2.1	0.86
GMM background subtraction (MOG2)	4.33	1.7	6.28
Lucas-Kanade optical flow	22.9 of 200 points moving	2.07 px/frame mean speed	2.53

Agreement: the frame-differencing and MOG2 masks overlapped with a mean IoU of 0.155, and 41.1% of the moving Lucas-Kanade points fell inside the MOG2 foreground.

Observations

- **No person, no motion:** in 188 of 1478 frame pairs (13%) both masks were practically empty. These are the moments when nobody is in view, so the still camera and the static room produced almost no false motion.
- **Masks:** frame differencing marked on average 2.07% of the frame and MOG2 4.33%. Differencing only marks the parts that changed since the previous frame, mainly the edges of the arms, legs and hair, so the plain black T-shirt is left hollow; MOG2 compares with the learned empty room and fills in much more of the body.
- **Peak:** the largest MOG2 value was 27.3% at 3.0 s, when the student was closest to the camera and covered a large part of the frame.
- **Lucas-Kanade:** about 23 of 200 tracked corners moved per frame, at 2.07 px/frame on average; the moving points (green arrows) lie on the student's body, while corners on the window and bed stay still. Only optical flow gives the direction and speed of the motion.
- **Speed:** frame differencing was the fastest (0.86 ms/frame), Lucas-Kanade 2.53 ms and MOG2 6.28 ms; all three run in real time at this resolution.

Table 24. *Characteristics of the three techniques (general properties, not measurements).*

Property	Frame differencing	GMM (MOG2)	Lucas-Kanade
Output	Changed pixels between 2 frames	Pixels unlike the learned background	Motion vector per tracked point
Memory of the past	Previous frame only	Statistical model of many frames	Previous frame only

Property	Frame differencing	GMM (MOG2)	Lucas-Kanade
Typical weakness	Hollow masks for uniform objects; misses objects that stop	Needs warm-up; absorbs objects that stay still; lighting changes	Sparse; loses points on fast motion or occlusion
Camera motion	Whole frame marked as moving	Whole frame marked as foreground	Measures it (global flow)
Gives direction/speed	No	No	Yes

8. Execution screenshots

The screenshots in this section are genuine captures of the project running on the computer described in Section 2. tools/capture_screenshots.py opened a real console window (Windows PowerShell), ran each command in it and, when the command had finished, captured that window with the Windows PrintWindow function. The unit runs used python main.py --unit N --screenshots, which shows the main result figures in live windows while the program runs and captures each window (window title 'figure displayed by main.py'). Only the program's own windows are captured. Screen: 1920x1008 px work area, 150% scaling.

Table 25. *Captured runs (outputs/screenshots_index.json).*

Step	Command	Captured	Exit code	Windows
Input check	python main.py --check-inputs	2026-11-01 20:41:22	0	0
Unit 3 run	python main.py --unit 3 --screenshots	2026-11-01 20:42:20	0	9
Unit 4 run	python main.py --unit 4 --screenshots	2026-11-01 20:42:42	0	4
Unit 5 run	python main.py --unit 5 --screenshots	2026-11-01 20:43:55	0	8
Automated tests	python -m pytest -v -p no:cacheprovider	2026-11-01 20:44:38	0	0

8.1 Input check

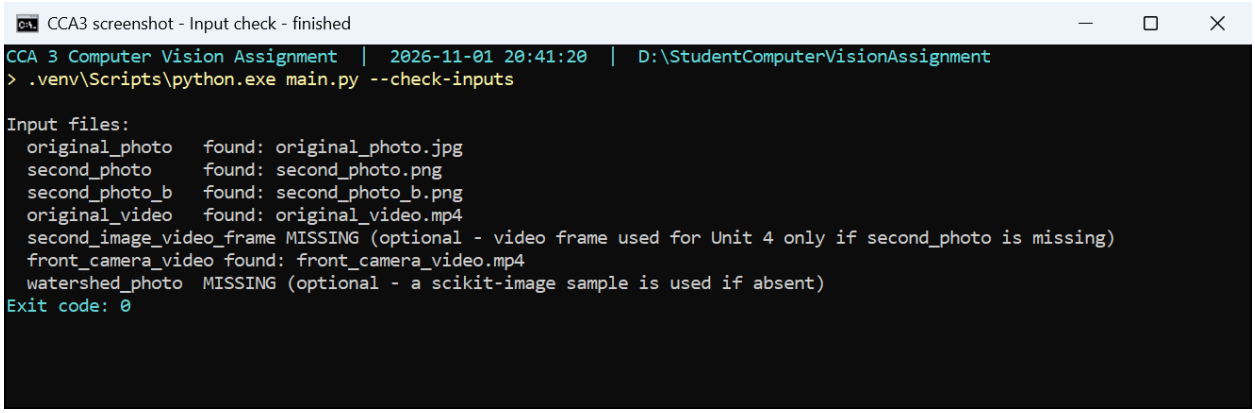


Figure 31. *Console screenshot - main.py --check-inputs lists which input files are present in the inputs folder. (captured 2026-11-01 20:41)* *[outputs/screenshot_terminal_check_inputs.png]*

8.2 Unit 3 - restoration and segmentation

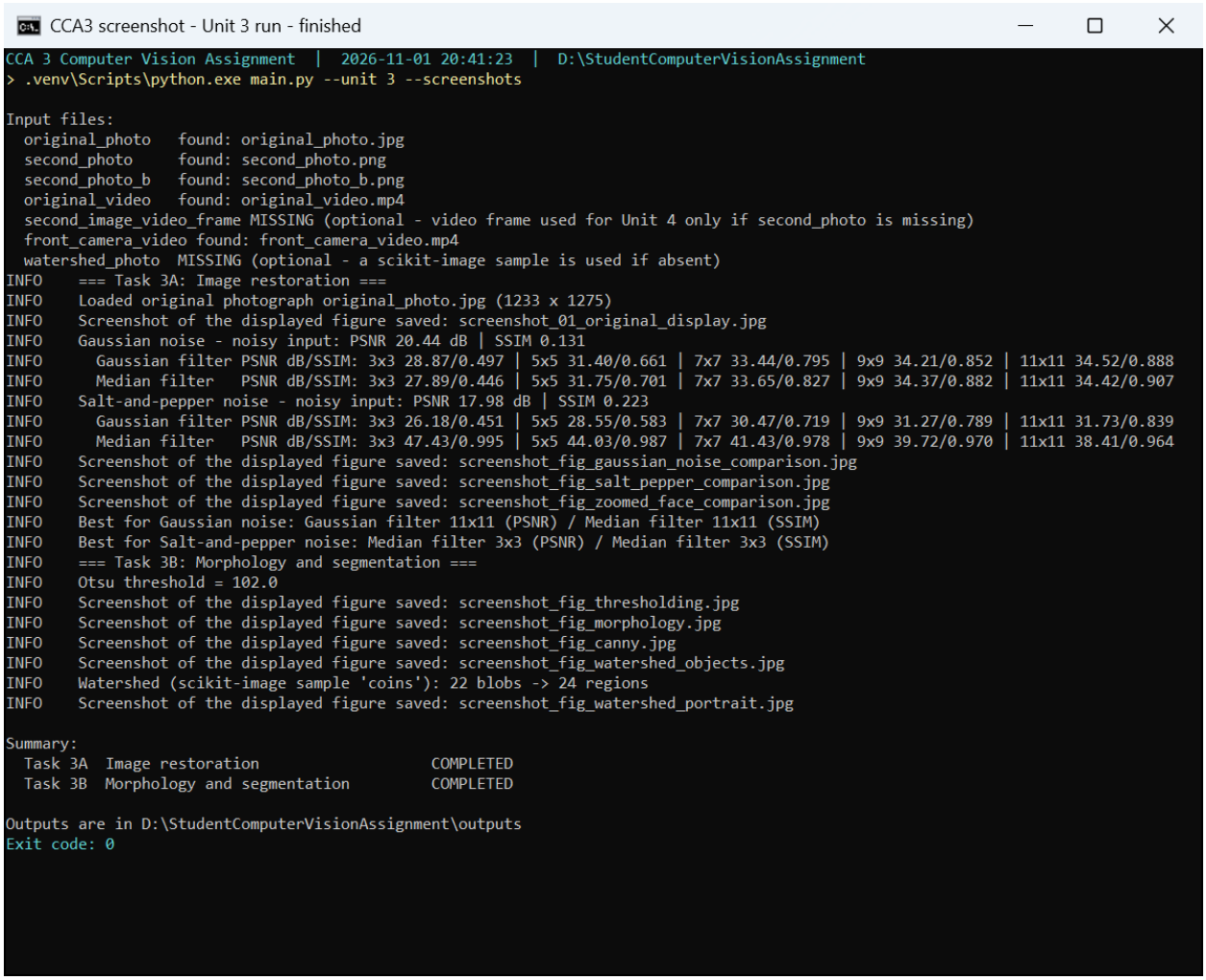


Figure 32. *Console screenshot - Run of Unit 3 (tasks 3A and 3B). The console shows the PSNR/SSIM of every filter and kernel size, the Otsu threshold, the watershed result and one line for each figure window that was captured. (captured 2026-11-01 20:42)*
[outputs/unit3/screenshot_terminal_unit3.png]

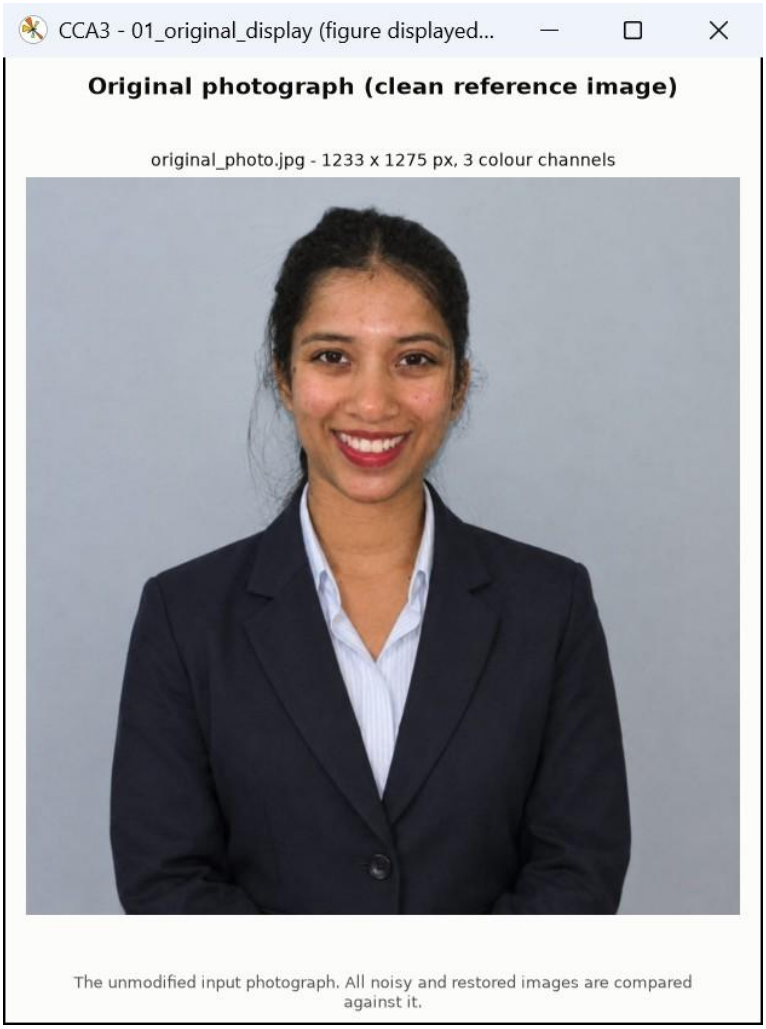


Figure 33. *Screenshot of the live figure window - the original input photograph as displayed by main.py at the start of Unit 3 (the clean reference image; size shown in the title).* *[outputs/unit3/restoration/screenshot_01_original_display.jpg]*

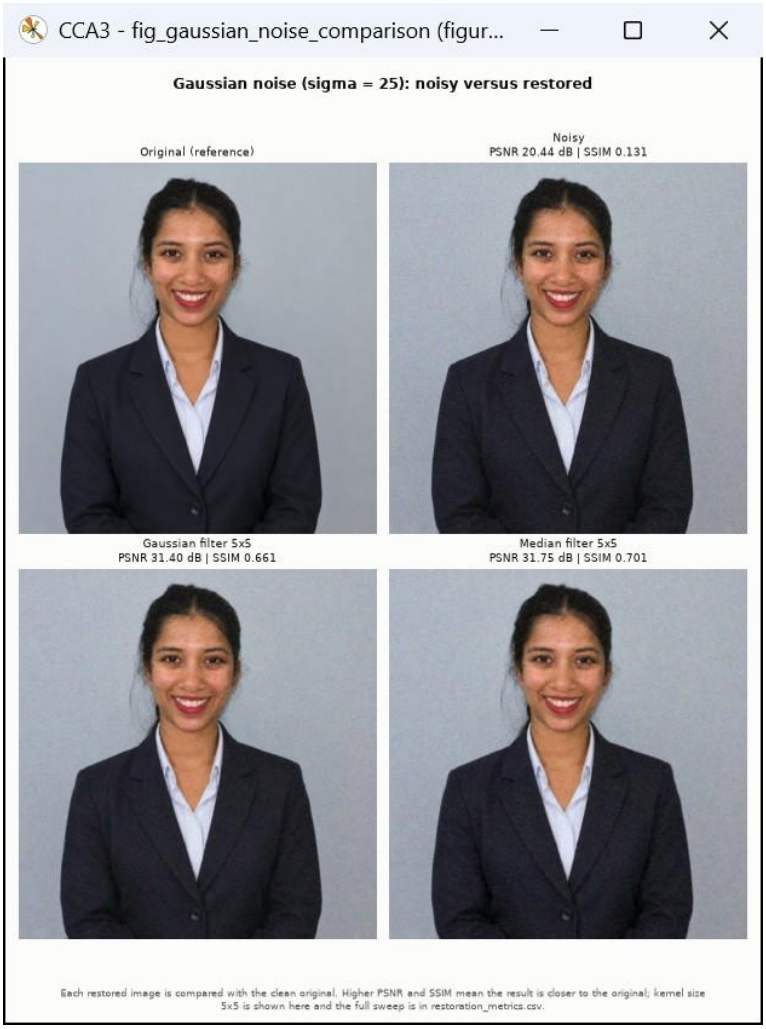


Figure 34. *Screenshot of the live figure window - Gaussian noise and its Gaussian- and median-filtered restorations; PSNR and SSIM against the original are printed above each panel.* *[outputs/unit3/restoration/screenshot_fig_gaussian_noise_comparison.jpg]*

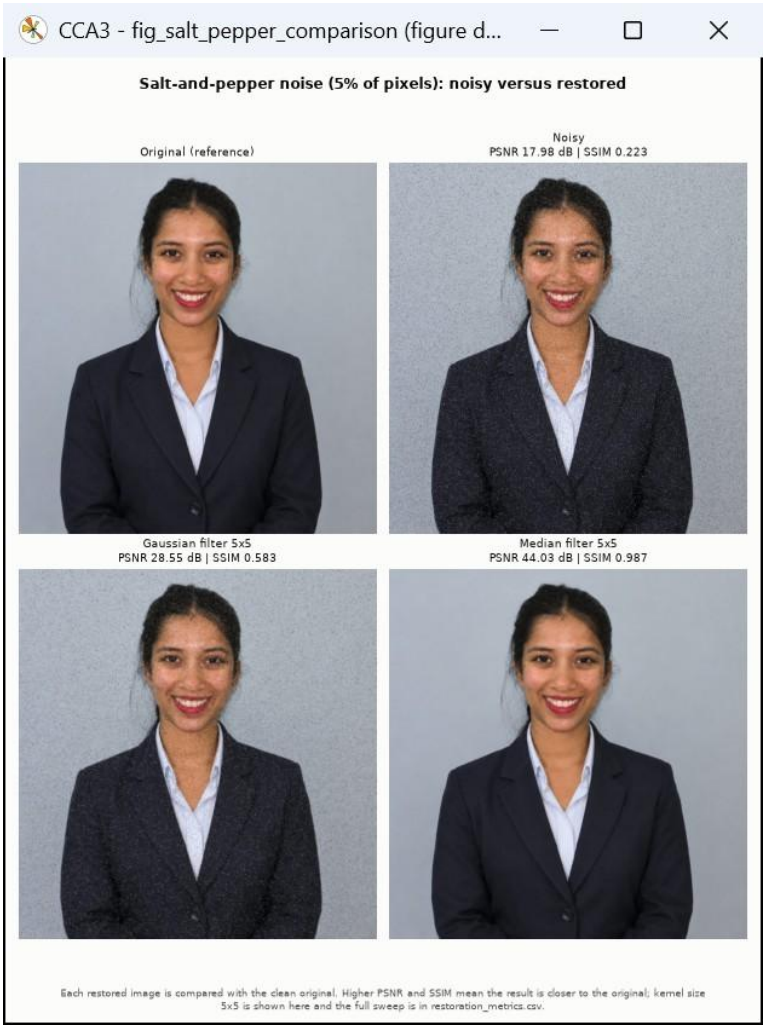


Figure 35. *Screenshot of the live figure window - salt-and-pepper noise and both restorations; the median filter removes the black and white dots almost completely.* *[outputs/unit3/restoration/screenshot_fig_salt_pepper_comparison.jpg]*

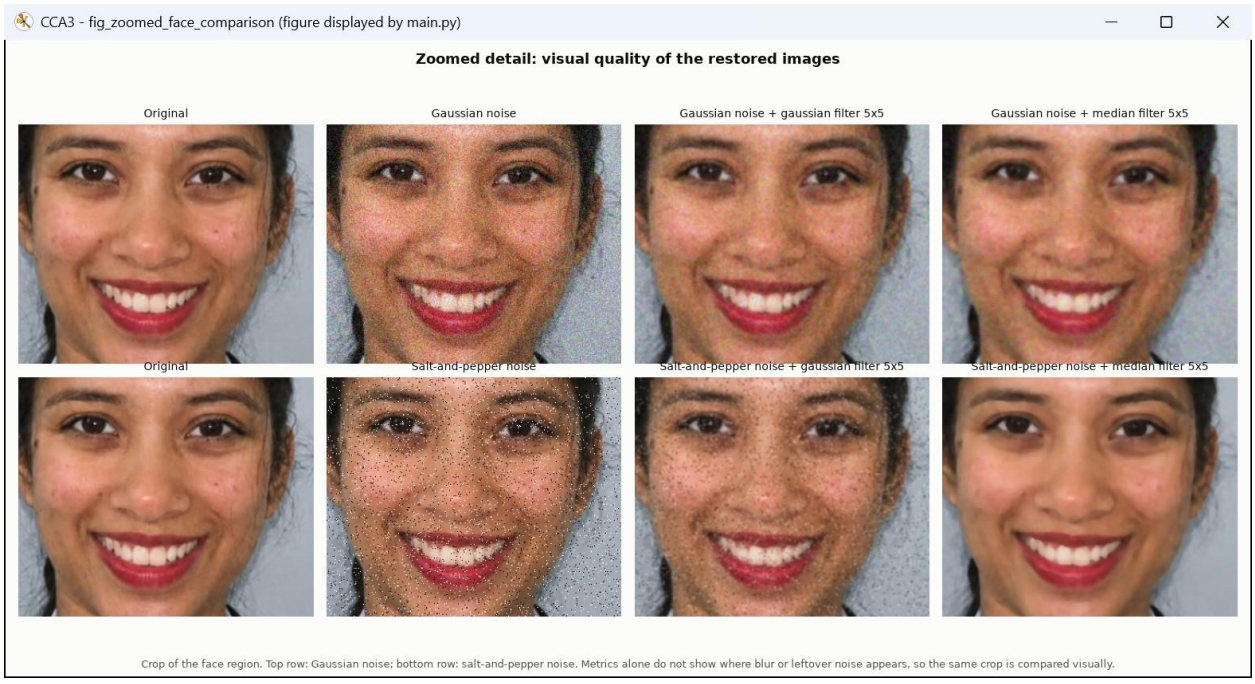


Figure 36. *Screenshot of the live figure window - zoomed face crops used to judge visual quality next to the metrics.*
[outputs/unit3/restoration/screenshot_fig_zoomed_face_comparison.jpg]

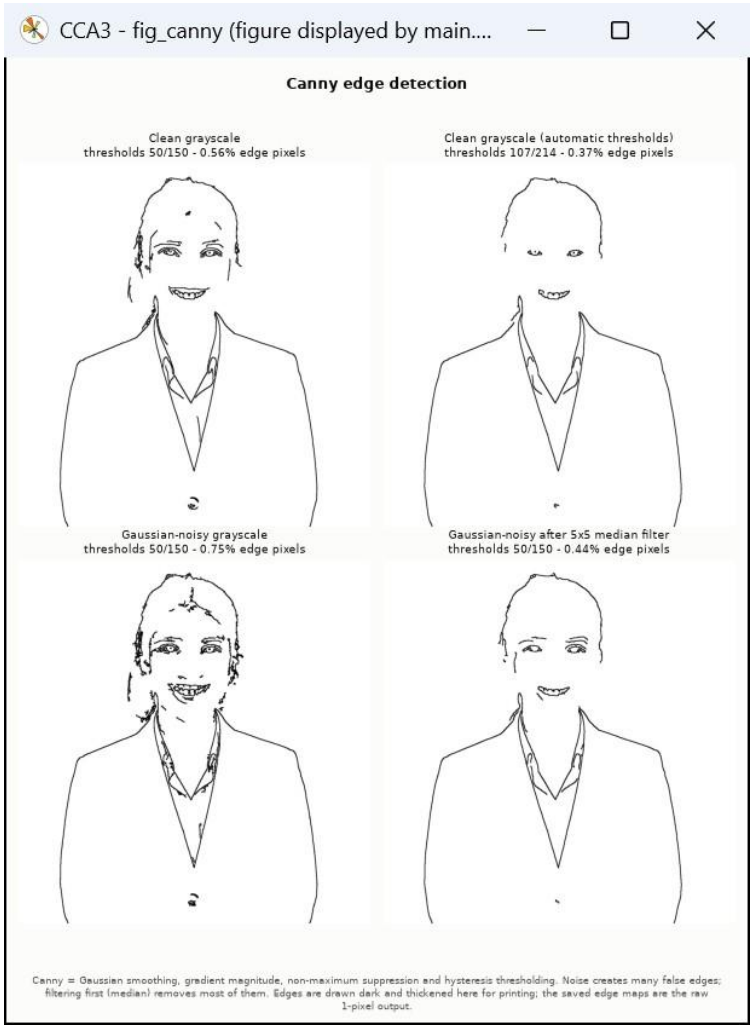


Figure 37. *Screenshot of the live figure window - Canny edges on the clean, noisy and median-restored portrait with edge-pixel percentages.* *[outputs/unit3/segmentation/screenshot_fig_canny.jpg]*

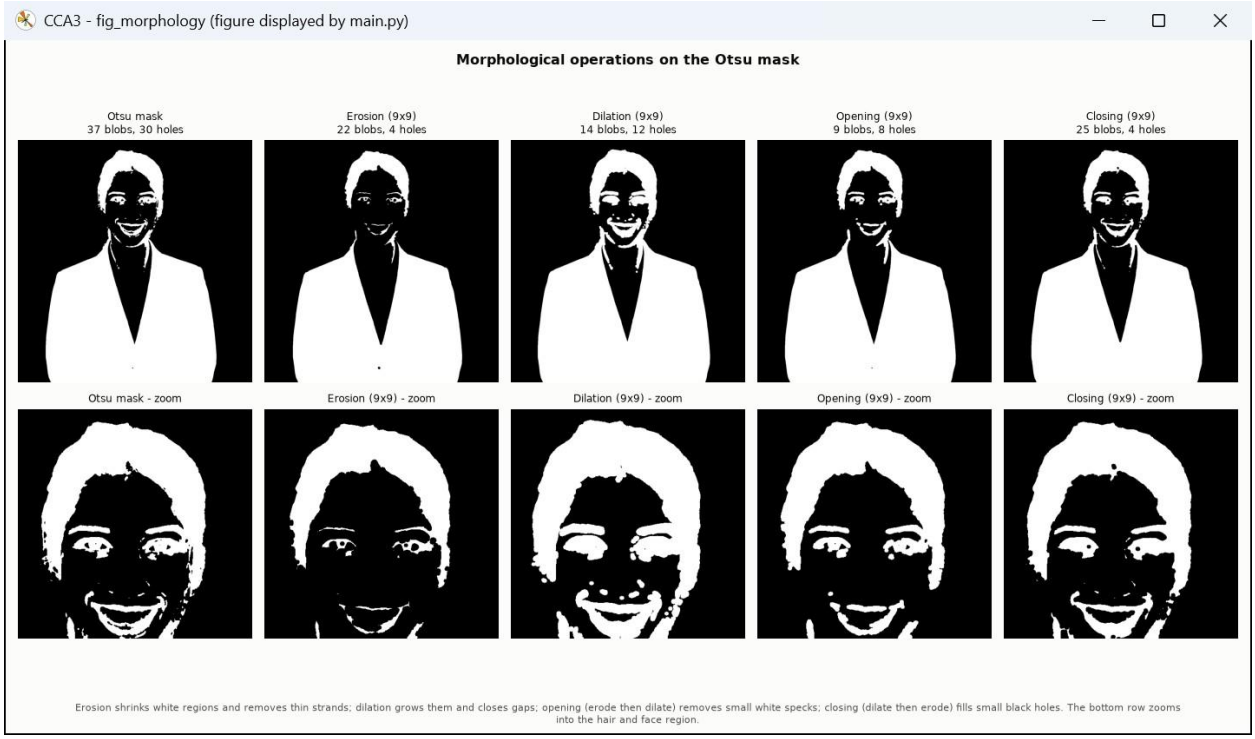


Figure 38. *Screenshot of the live figure window - erosion, dilation, opening and closing of the Otsu mask, with blob and hole counts.* *[outputs/unit3/segmentation/screenshot_fig_morphology.jpg]*

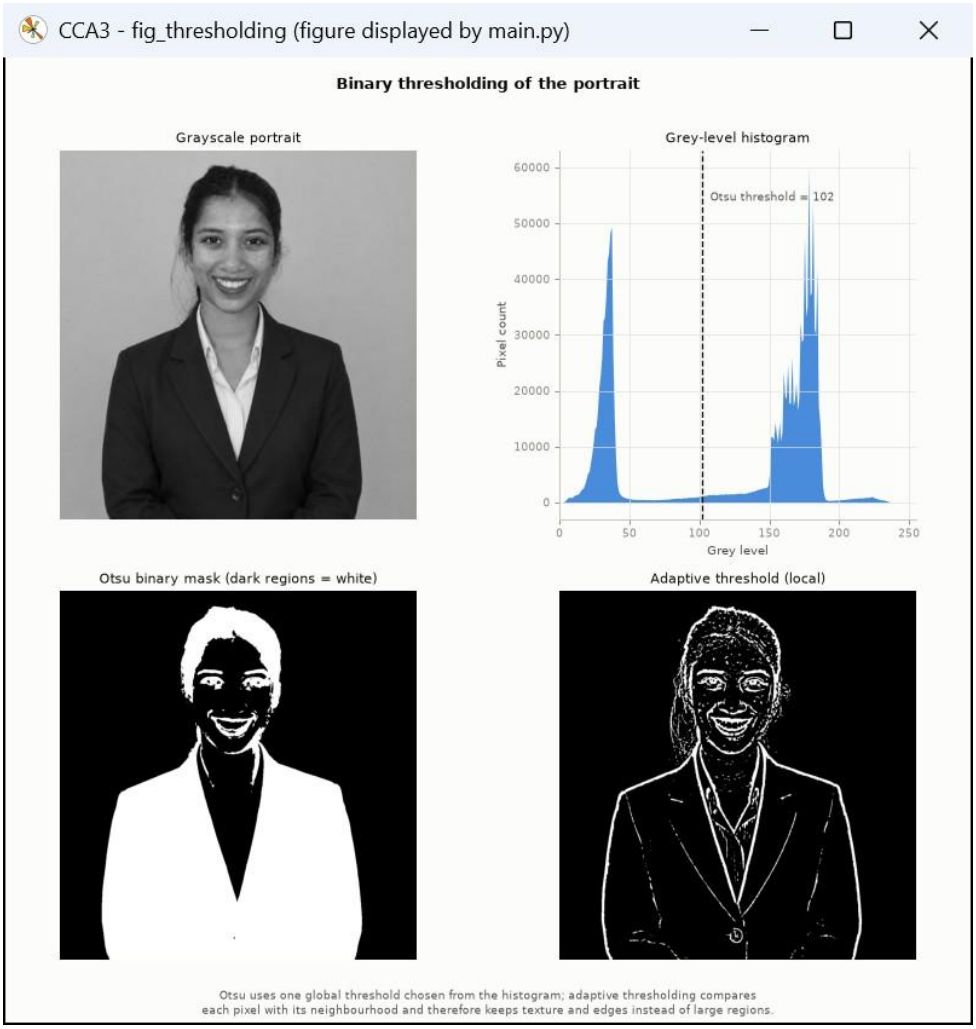


Figure 39. *Screenshot of the live figure window - grayscale portrait, histogram with the Otsu threshold, and the Otsu and adaptive binary masks.* *[outputs/unit3/segmentation/screenshot_fig_thresholding.jpg]*

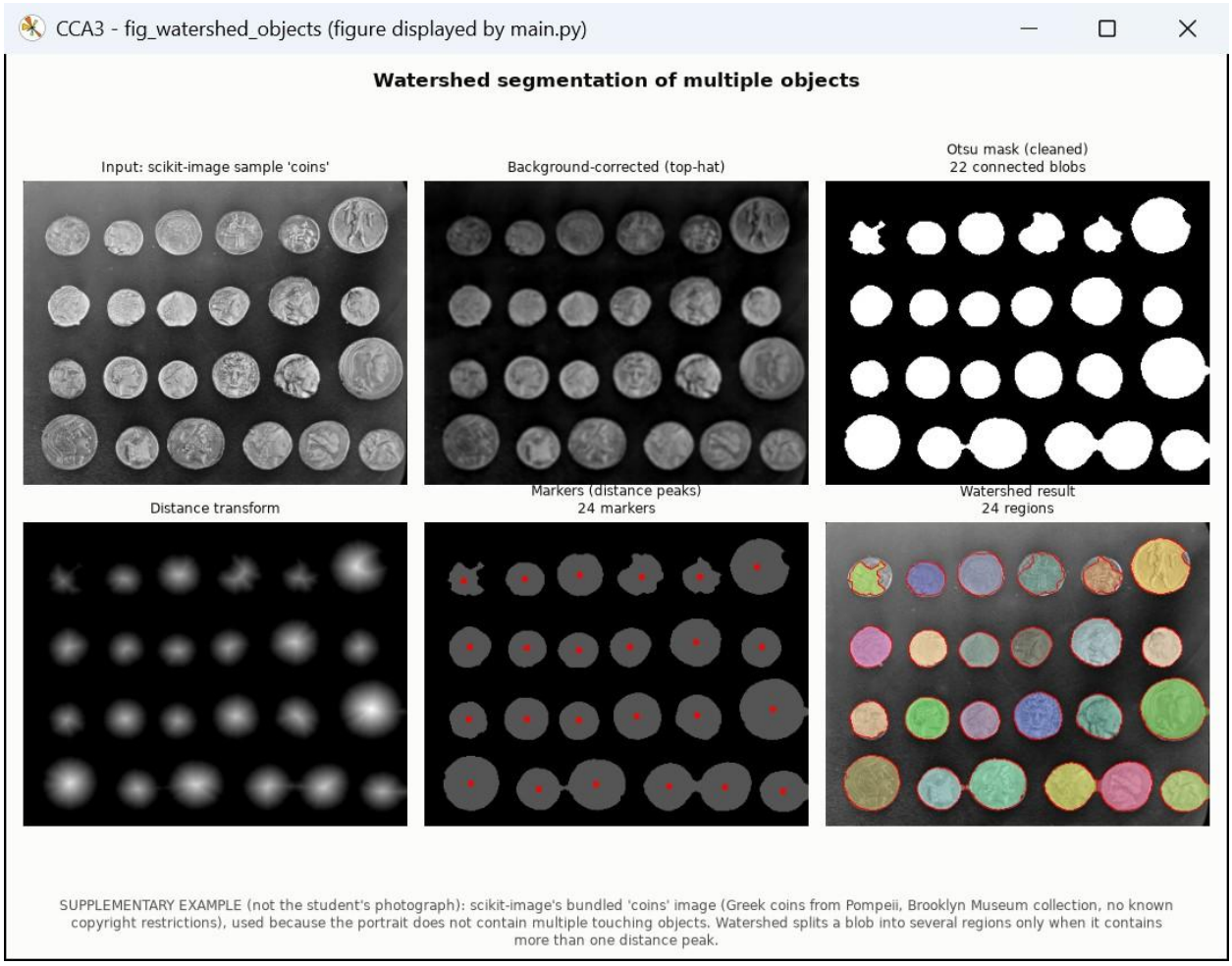


Figure 40. *Screenshot of the live figure window - distance-transform watershed on the supplementary coins image (input, background correction, mask, distance map, markers, regions).* *[outputs/unit3/segmentation/screenshot_fig_watershed_objects.jpg]*

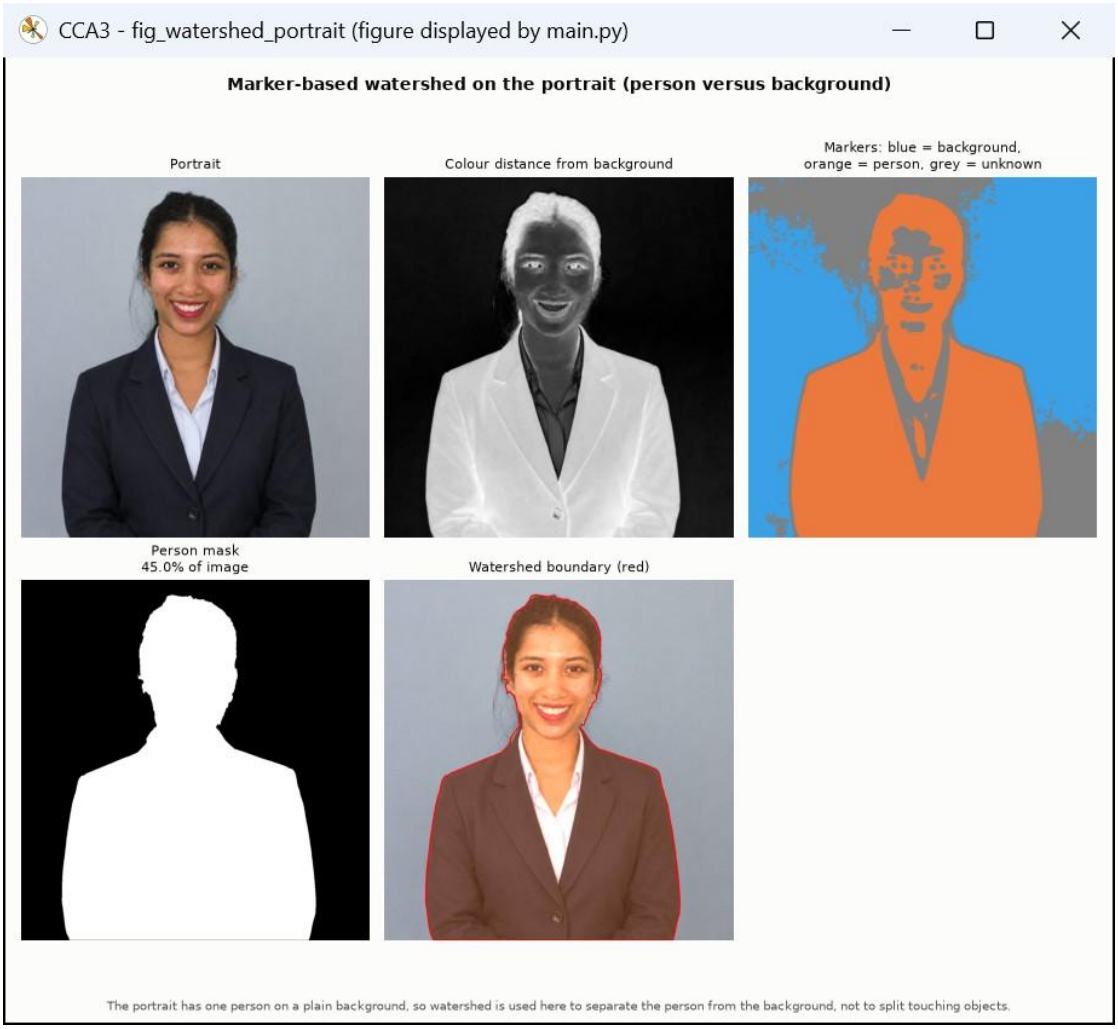


Figure 41. *Screenshot of the live figure window - marker-based watershed separating the person from the backdrop.*
[outputs/unit3/segmentation/screenshot_fig_watershed_portrait.jpg]

8.3 Unit 4 - HOG and SIFT

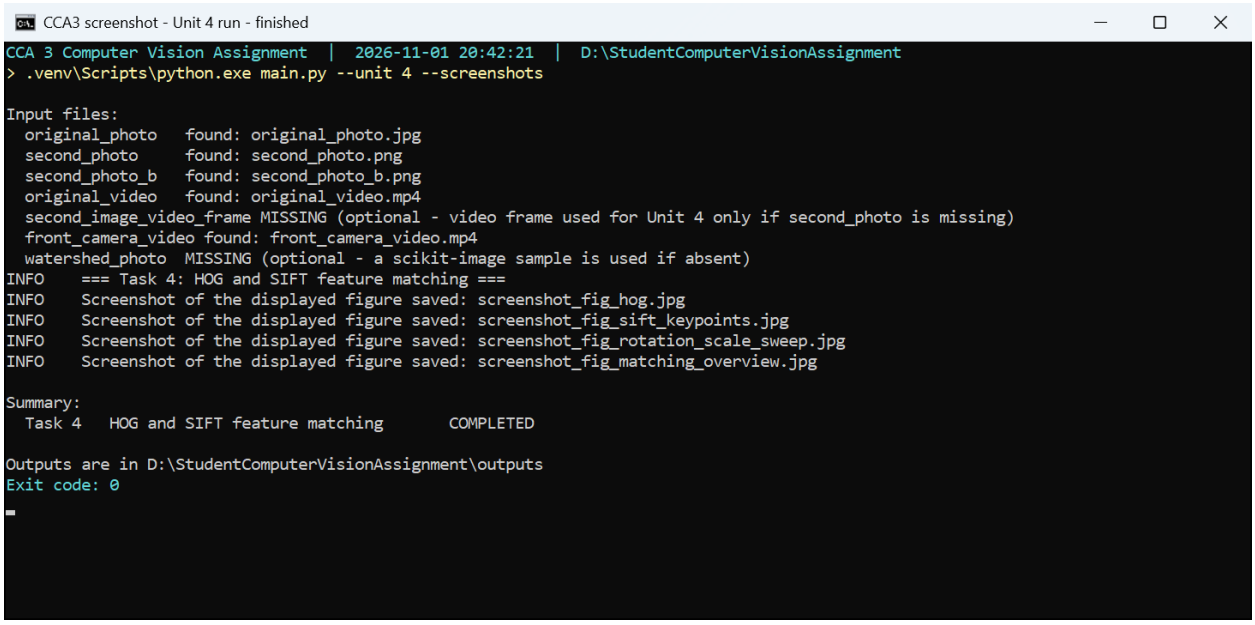


Figure 42. *Console screenshot - Run of Unit 4. The warning confirms that the second photograph is missing, so the two-photo experiment is reported as pending while the HOG, SIFT and transformation experiments complete. (captured 2026-11-01 20:42)*
[outputs/unit4/screenshot_terminal_unit4.png]

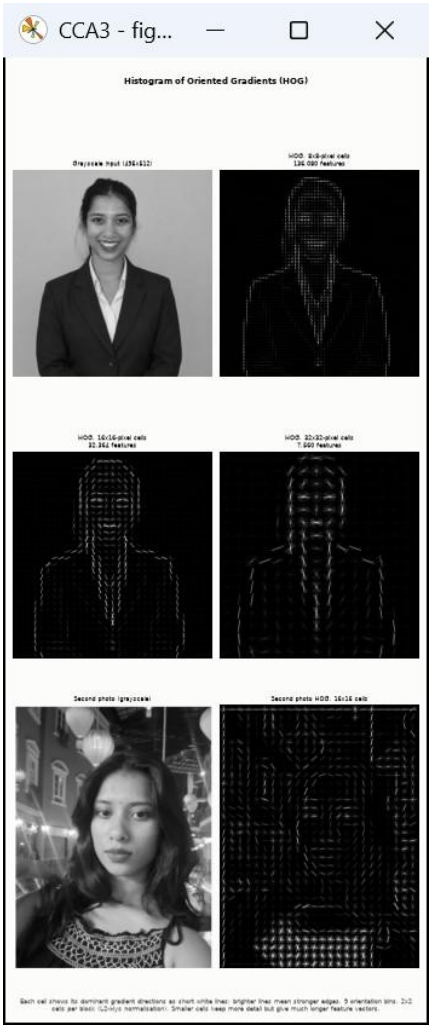


Figure 43. *Screenshot of the live figure window - HOG visualisations for 8, 16 and 32-pixel cells with feature-vector lengths.*
[outputs/unit4/screenshot_fig_hog.jpg]

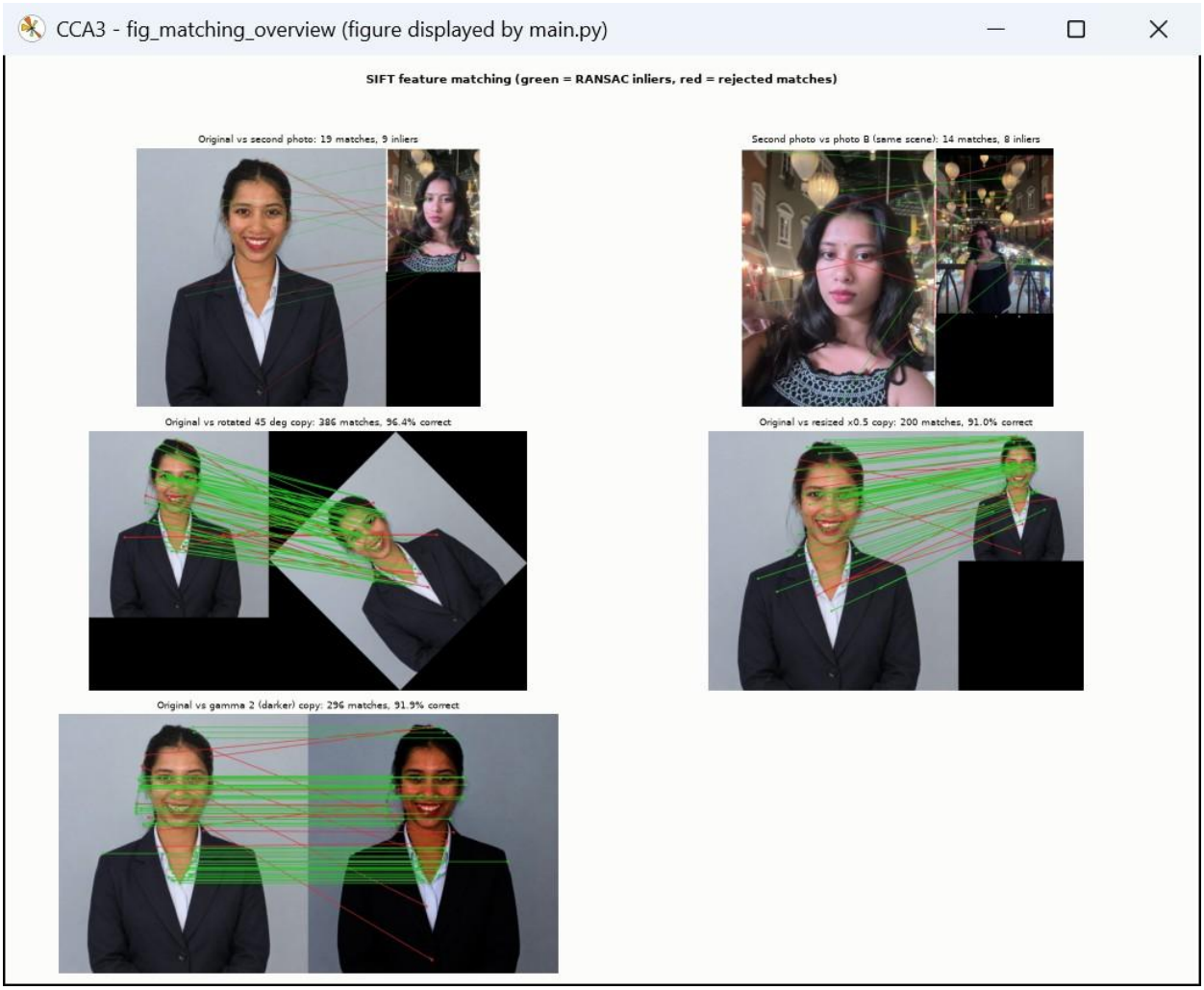


Figure 44. *Screenshot of the live figure window - SIFT matches (green = RANSAC inliers, red = rejected) for the image pairs and the rotated, resized and darkened copies.* *[outputs/unit4/screenshot_fig_matching_overview.jpg]*

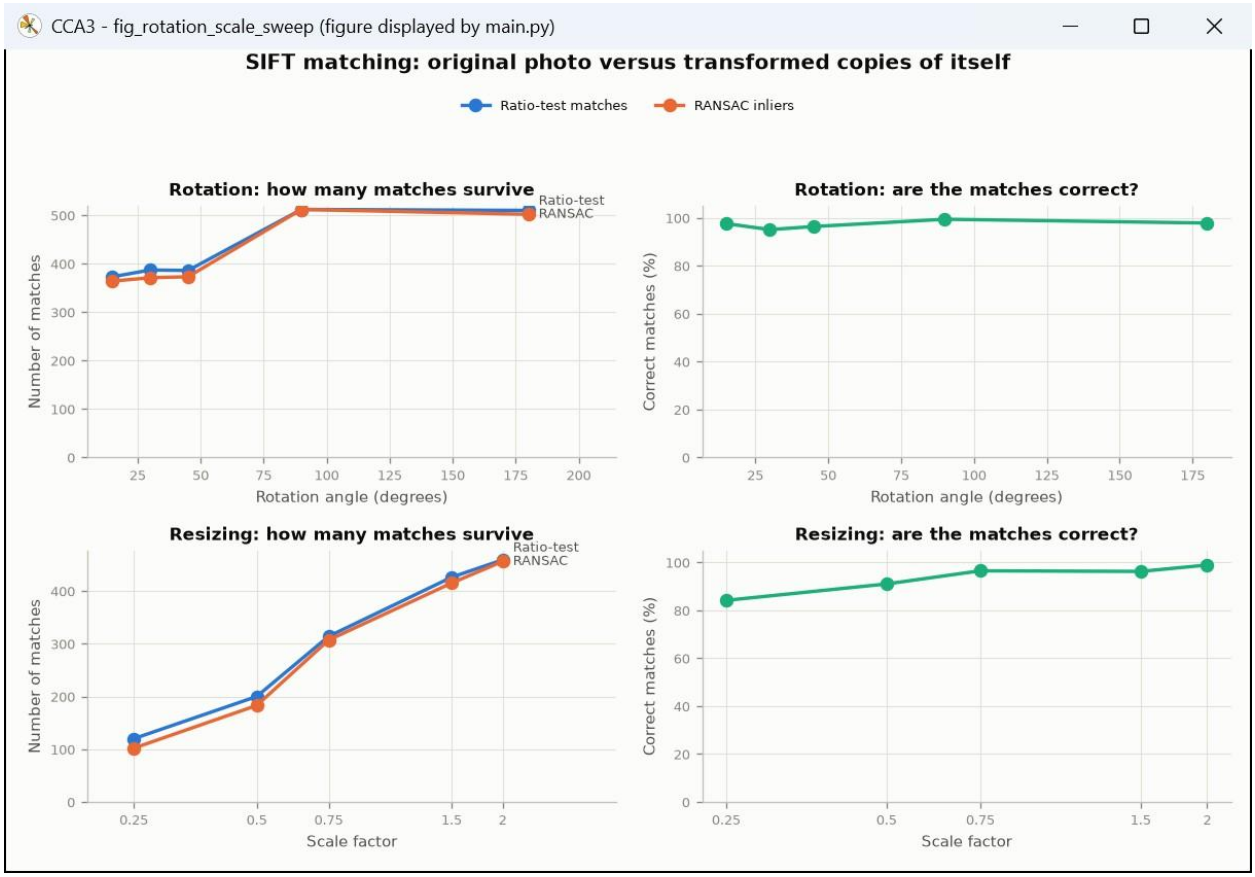


Figure 45. *Screenshot of the live figure window - number and correctness of SIFT matches against rotation angle and scale.*
[outputs/unit4/screenshot_fig_rotation_scale_sweep.jpg]

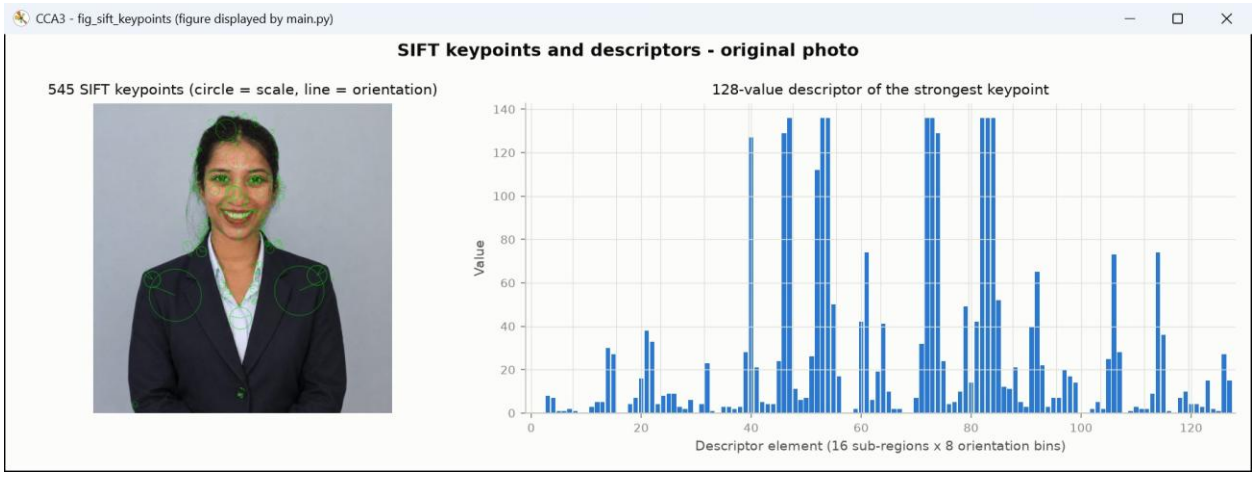


Figure 46. *Screenshot of the live figure window - SIFT keypoints on the portrait and the 128-value descriptor of the strongest one.*
[outputs/unit4/screenshot_fig_sift_keypoints.jpg]

8.4 Unit 5 - Haar cascade, YOLO and motion

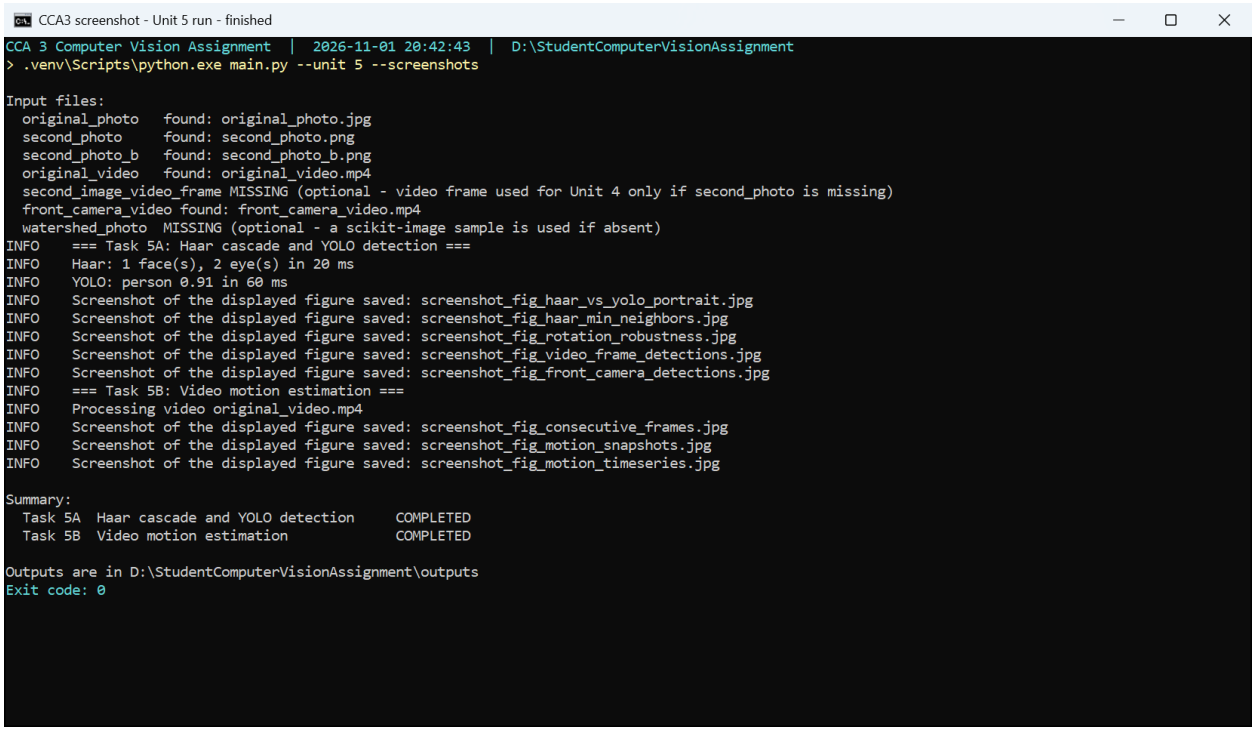


Figure 47. *Console screenshot - Run of Unit 5. Haar and YOLO results are printed with timings; task 5B reports the missing video as pending instead of crashing. (captured 2026-11-01 20:43)* *[outputs/unit5/screenshot_terminal_unit5.png]*

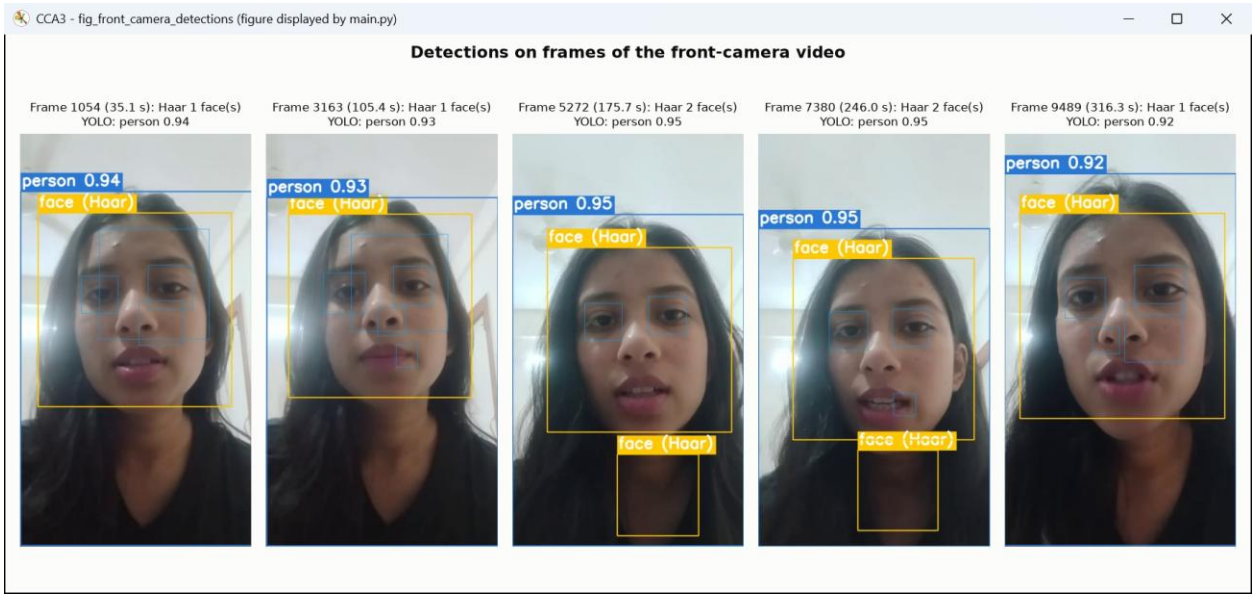


Figure 48. *Screenshot of the live figure window - Haar faces/eyes and YOLO on frames of the front-camera video, including the false face and eye boxes.* *[outputs/unit5/detection/screenshot_fig_front_camera_detections.jpg]*

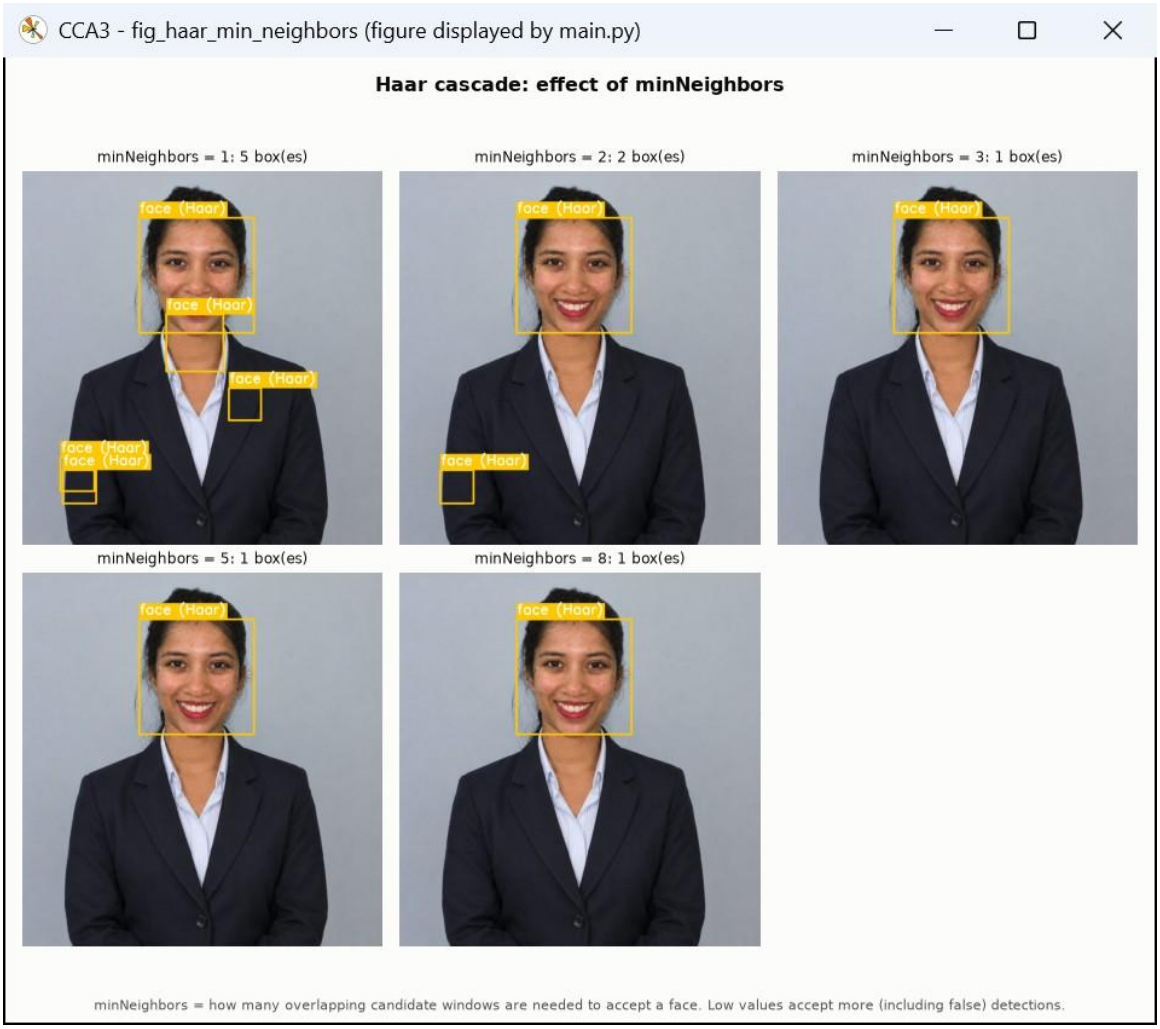


Figure 49. *Screenshot of the live figure window - Haar detections for minNeighbors 1-8, showing false boxes at low values.*
[outputs/unit5/detection/screenshot_fig_haar_min_neighbors.jpg]

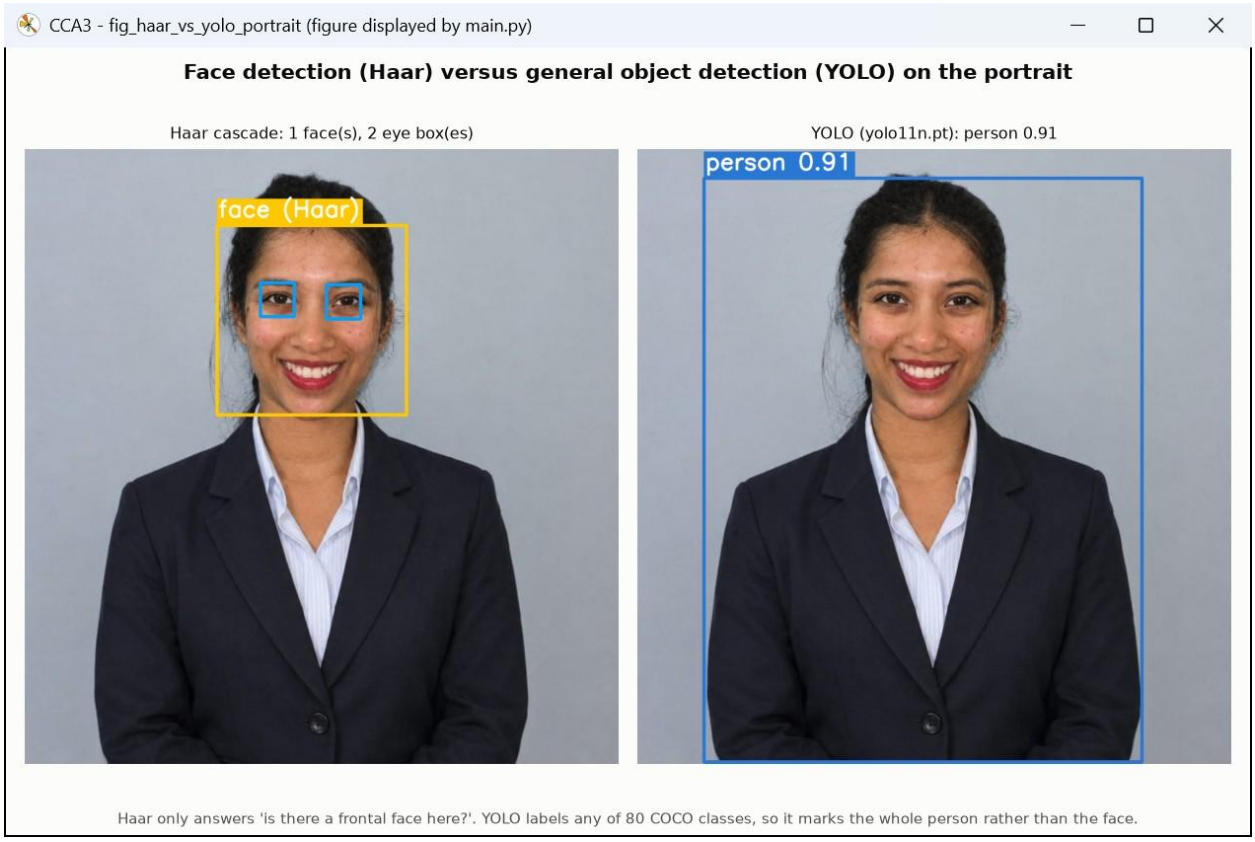


Figure 50. *Screenshot of the live figure window - Haar face/eye boxes next to the YOLO 'person' box with its confidence.*
[outputs/unit5/detection/screenshot_fig_haar_vs_yolo_portrait.jpg]

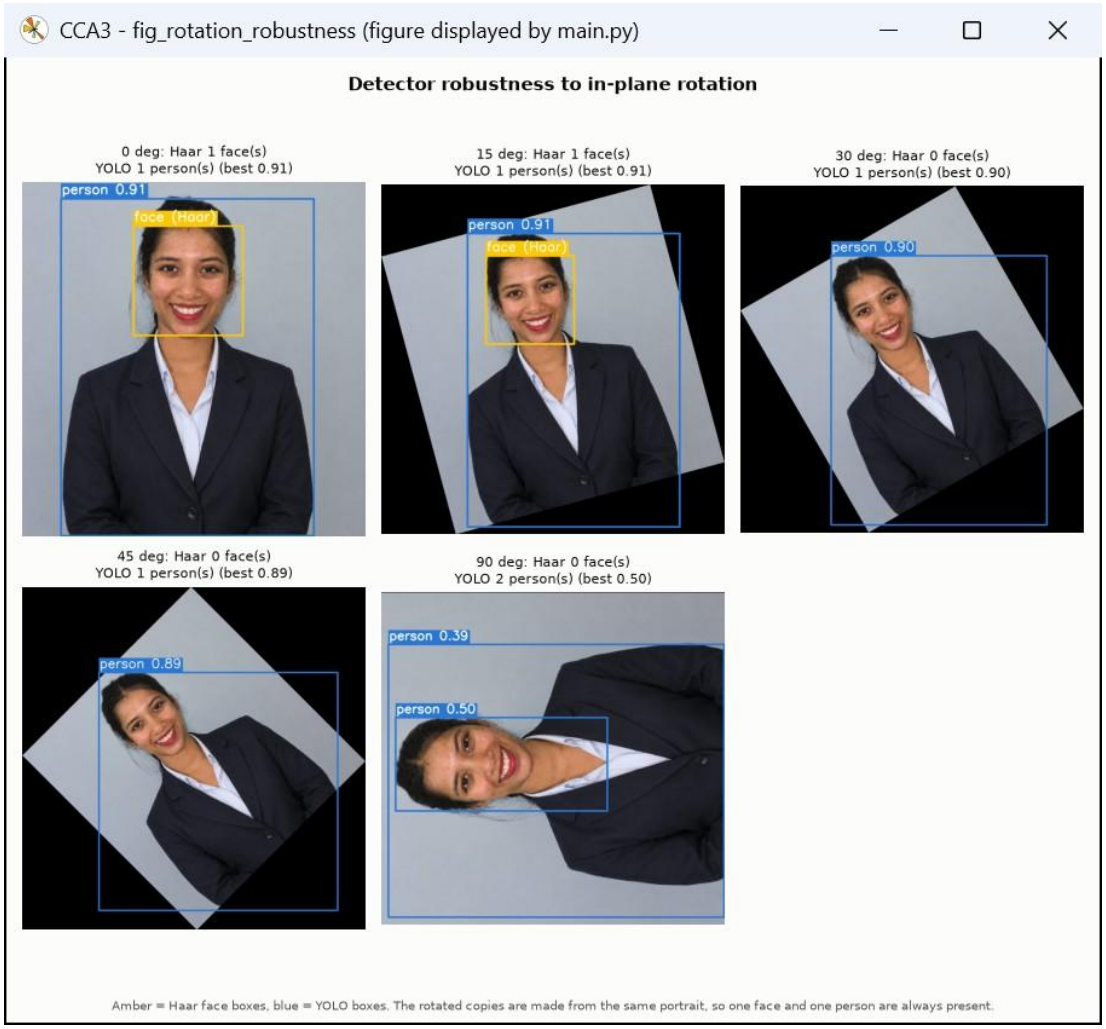


Figure 51. *Screenshot of the live figure window - Haar and YOLO on rotated copies; the Haar face box disappears from 30 degrees.*
[outputs/unit5/detection/screenshot_fig_rotation_robustness.jpg]



Figure 52. *Screenshot of the live figure window - Haar and YOLO on frames sampled from the self-recorded video.*
[outputs/unit5/detection/screenshot_fig_video_frame_detections.jpg]

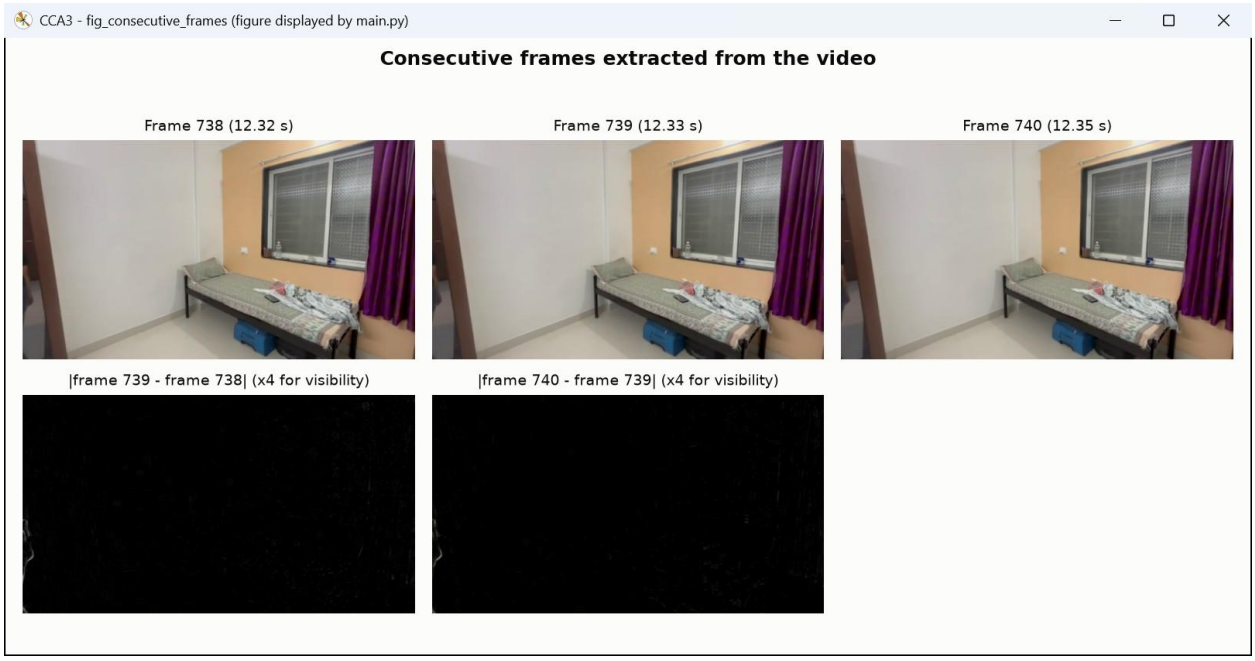


Figure 53. *Screenshot of the live figure window - consecutive frames extracted from the video and their differences.*
[outputs/unit5/motion/screenshot_fig_consecutive_frames.jpg]

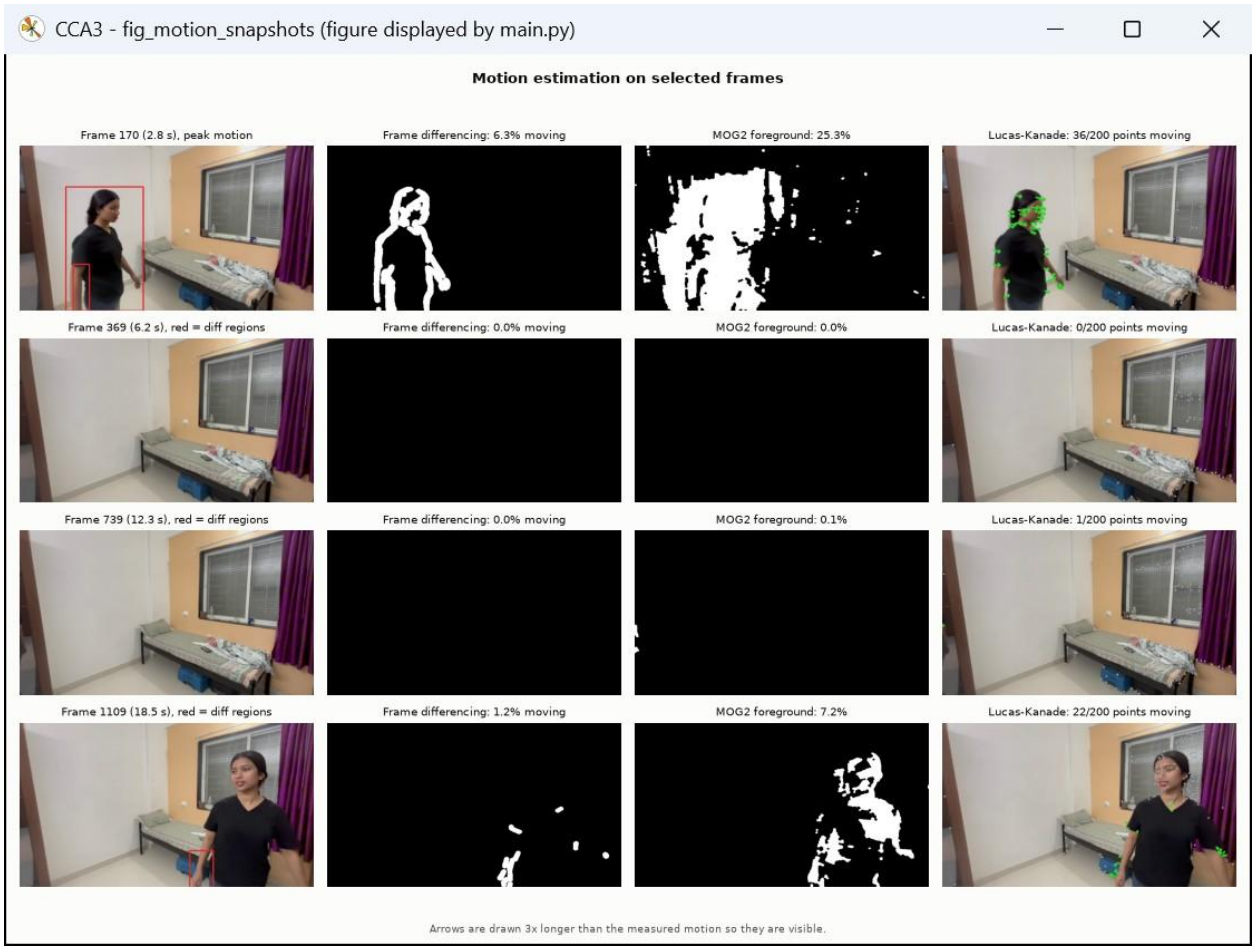


Figure 54. *Screenshot of the live figure window - frame differencing mask, MOG2 foreground mask and Lucas-Kanade vectors.*
[outputs/unit5/motion/screenshot_fig_motion_snapshots.jpg]

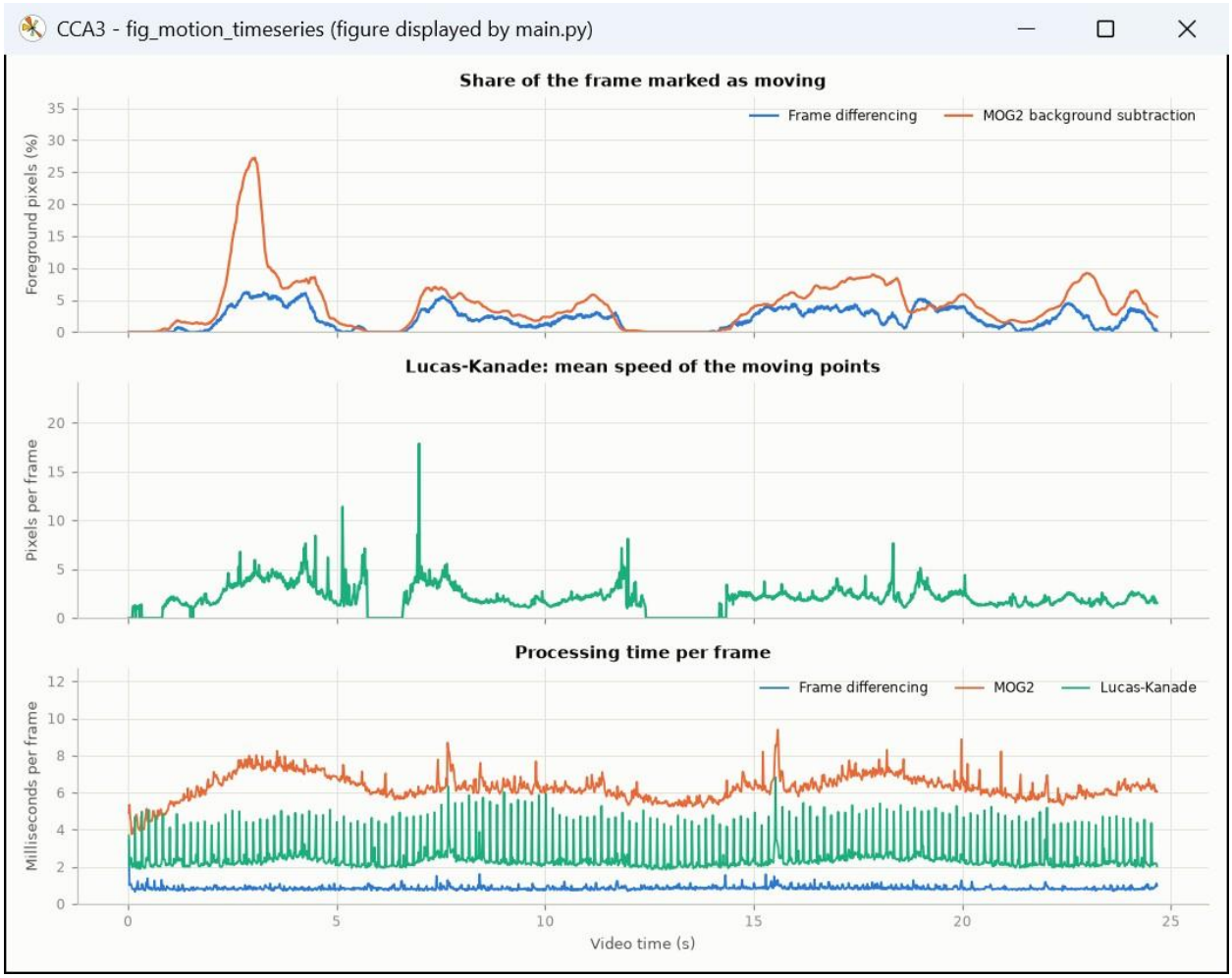


Figure 55. *Screenshot of the live figure window - per-frame foreground share, Lucas-Kanade speed and processing time.*
[outputs/unit5/motion/screenshot_fig_motion_timeseries.jpg]

8.5 Automated tests

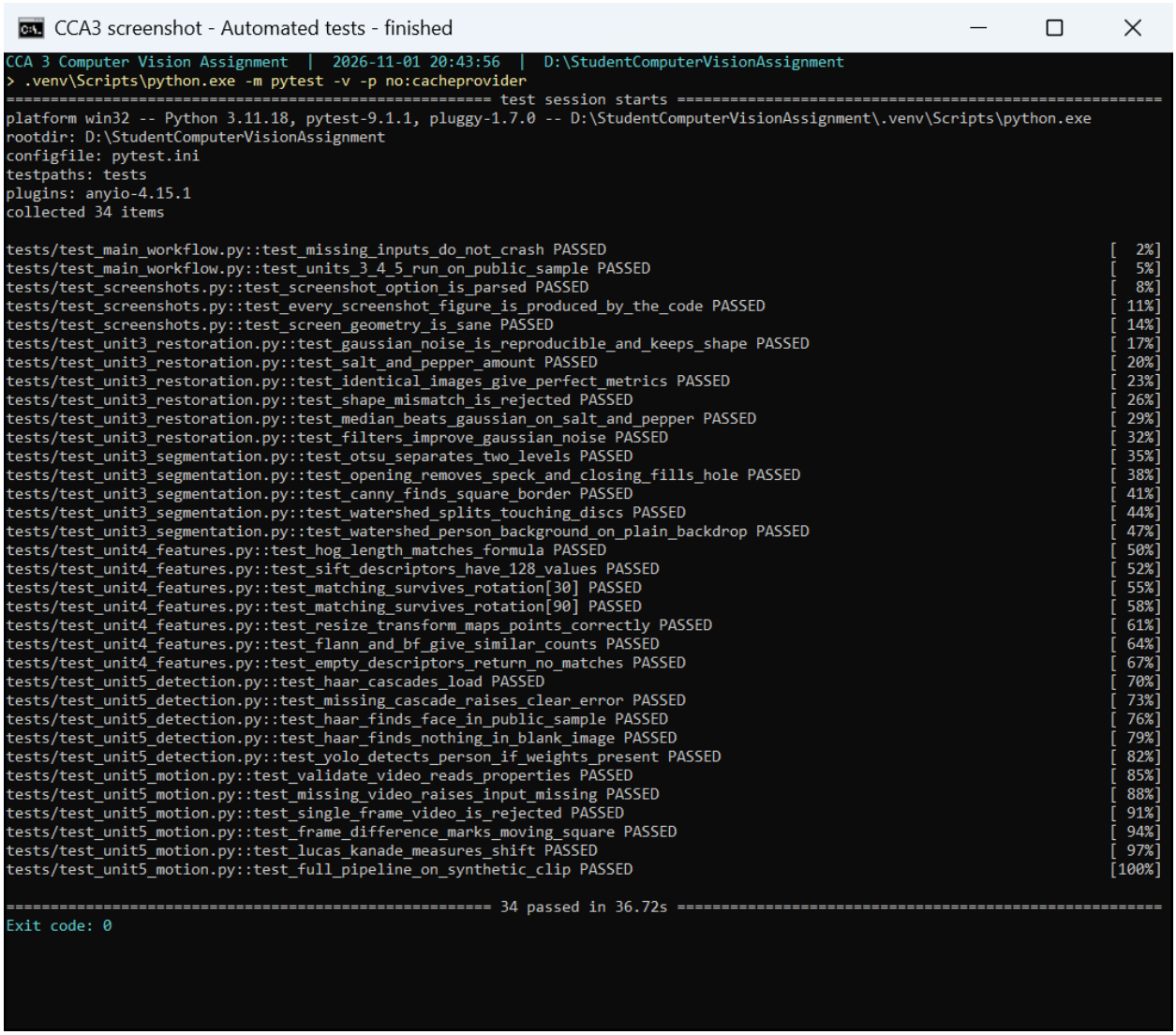


Figure 56. *Console screenshot - The automated test suite (pytest -v): every test name and its result. (captured 2026-11-01 20:44) [outputs/screenshot_terminal_pytest.png]*

9. Limitations and pending work

- PSNR and SSIM measure similarity to the reference image, not perceived quality; the visual comparisons should be read together with the numbers.
- The coins image for watershed is a supplementary public image; adding your own photo of touching objects as inputs/watershed_photo.jpg makes the experiment use your own data automatically.
- Rotation, scaling and lighting experiments use transformed copies of one photograph, which isolates each effect but does not reproduce real 3-D viewpoint changes.
- Timings were measured on a laptop CPU (no GPU) and vary between runs.

10. Conclusion

Restoration: the median filter is the best choice for salt-and-pepper noise (47.43 dB with a 3x3 kernel). For Gaussian noise the Gaussian and median filters performed almost equally (best PSNR 34.52 vs 34.42 dB), and the kernel size must balance noise removal against loss of detail.

Segmentation: Otsu thresholding separated the dark hair and blazer from the light background and skin; opening and closing cleaned the mask; Canny showed that noise must be filtered before edge detection; watershed split

touching objects using distance-transform markers and separated the person from the backdrop with colour-based markers.

Features: HOG captured the overall shape of the portrait, and SIFT matching stayed accurate under rotation (at least 95% correct matches), scaling and lighting changes; the number of matches drops mainly when the image becomes small. Matching the portrait with the second photograph gave no reliable correspondences, because the two photos share almost no local texture (different scene, clothes and lighting); even two low-resolution photos of the same scene matched poorly.

Recognition: the Haar cascade found 1 face on the upright portrait, missed it at 30, 45, 90 degrees of rotation, produced false boxes with minNeighbors 1, 2. YOLO found the person at every angle but returned a duplicate box at 90 degrees.

Motion: on the self-recorded video MOG2 gave the most complete foreground masks (4.33% of the frame on average versus 2.07% for frame differencing, which leaves the uniform T-shirt hollow), frame differencing was the fastest, and Lucas-Kanade was the only method that gave the direction and speed of the motion.

11. Git repository and explanation video

Git repository URL: https://github.com/Kritikakumrawat/computer-vision-cca3

Branch: main (GitHub account: github.com/Kritikakumrawat).

Project explanation video (Google Drive):
https://drive.google.com/file/d/1KC3AiaGDgUWXWRE5pkoV7XZHZk3L9u9n/view?usp=drivesdk

Access for the instructor must be granted on both the repository and the Google Drive folder (see docs/SUBMISSION_CHECKLIST.md).

Appendix A - Source code

The complete source code is in the project folder (and the Git repository). Each task is one module:

Table 26. *Source files.*

File	Contents
main.py	Command-line entry point: --unit 3/4/5, --all, --check-inputs, --skip-yolo
src/config.py	All file names and parameters (noise levels, kernel sizes, thresholds, model name)
src/utils.py	Input validation, image I/O, figures, JSON/CSV writing, timer
src/image_restoration.py	Noise models, Gaussian/median filters, PSNR/SSIM, restoration figures
src/segmentation.py	Otsu/adaptive thresholding, morphology, Canny, watershed (objects and portrait)
src/feature_extraction.py	HOG, SIFT, BF/FLANN matching, RANSAC, rotation/scale/lighting experiments
src/object_detection.py	Haar cascade face/eye detection, YOLO detection, robustness experiments
src/motion_estimation.py	Video validation, frame differencing, MOG2, Lucas-Kanade, output video/GIF
src/screen_capture.py	Windows window capture (PrintWindow) used for the execution screenshots
tests/	Automated tests (pytest) on synthetic and public sample data
tools/capture_screenshots.py	Runs each unit in a real console window and captures the screenshots
tools/extract_video_frame.py	Extracts and documents the clearest face frame from the student's video
tools/build_report.py	Builds this report from the outputs
tools/export_pdf.py	Updates the contents page and exports the PDF with Microsoft Word

Appendix B - How to reproduce

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python main.py --all
python -m pytest
python tools/capture_screenshots.py
python tools/build_report.py
python tools/export_pdf.py
```